



UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

LUCRARE DE LICENȚĂ

COORDONATOR:
Lector Dr. Popovici Adriana

ABSOLVENT:
Preda Slavoliub-Denis

TIMIȘOARA
2026

UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

**Dezvoltarea unei platforme web de management
al proiectelor software cu funcționalități asistate
de inteligența artificială**

COORDONATOR:
Lector Dr. Popovici Adriana

ABSOLVENT:
Preda Slavoliub-Denis

TIMIȘOARA
2026

Abstract

The software application presented in this thesis is called *SDLC Hub* and represents a web platform for software project management. The platform was developed for teams that need to organize requirements, tasks, sprints, boards, documentation, reports and technical activity in a single workspace. The central premise is that a project management tool should not be only a task list, but a connected environment in which planning, execution, collaboration, traceability and reporting support one another.

SDLC Hub supports Scrum, Kanban and Scrumban. The selected methodology is not handled as a simple label: it changes the available pages, editing rules, estimation fields, backlog and sprint behavior, board workflow and reports. Methodology transitions are controlled through impact previews, blockers, warnings and audit records.

A central contribution of this thesis is the integration of AI-assisted features into concrete project workflows. The current implementation is local-first and uses Ollama with the `qwen2.5-coder:7b` model. The application includes a methodology advisor, role suggestions, specification refinement, story point estimation, risk analysis, workload balancing, release note generation and automatic documentation from completed work. These functions provide editable recommendations, while the final decision remains with the user.

The platform also includes project roles and permissions, tasks and subtasks, comments, audit logs, board and backlog views, calendar events, notifications, internal documentation, delivery reports, PDF export, GitHub integration, WebSocket-based realtime updates, account management and a global administration console. The backend is implemented with FastAPI, SQLAlchemy, Alembic and PostgreSQL, while the frontend uses Next.js, React, TypeScript and Tailwind CSS. The thesis presents the motivation, existing solutions, requirements, design, implementation, user interface, testing results, limitations and future development directions.

Cuprins

1	Introducere	4
1.1	Contextul și problema abordată	4
1.2	Motivație	4
1.3	Descrierea informală a soluției	5
1.4	Scop și obiective	5
1.5	Contribuțiile autorului și rezultate	6
1.6	Structura lucrării	6
2	Prezentarea problematicii abordate	7
2.1	Managementul proiectelor software	7
2.2	Scrum, Kanban și Scrumban	7
2.3	Inteligența artificială în dezvoltarea software	7
2.4	Jira Software	8
2.5	Trello	9
2.6	GitHub Projects	9
2.7	Comparație obiectivă	10
3	Detalierea contribuției autorului	12
3.1	Analiza și proiectarea aplicației	12
3.1.1	Actori și roluri	12
3.1.2	Cerințe funcționale	12
3.1.3	Cerințe nefuncționale	13
3.1.4	Cazuri de utilizare	14
3.1.5	Arhitectura sistemului	23
3.1.6	Modelul datelor	25
3.1.7	API și interfețe externe	25
3.1.8	Fluxuri importante	26
3.1.9	Exemplu de utilizare urmărit	26
3.2	Implementarea aplicației	27
3.2.1	Tehnologii și justificarea alegerilor	27
3.2.2	Organizarea backend-ului	27
3.2.3	Organizarea frontend-ului	27
3.2.4	Baza de date și migrațiile	28
3.2.5	Autentificare, permisiuni și securitate	28
3.2.6	Managementul proiectelor și task-urilor	28

3.2.7	Funcționalități asistate de inteligența artificială	29
3.2.8	Comunicarea realtime prin WebSocket	31
3.2.9	Integrarea GitHub Webhooks	32
3.2.10	Calendar, notificări și activitate	34
3.2.11	Rapoarte, documentație și Global Admin	34
3.2.12	Manual de implementare și instalare	35
3.3	Interfața și utilizarea aplicației	36
3.3.1	Principii de interfață și navigare	36
3.3.2	Înregistrare și autentificare	37
3.3.3	Crearea proiectului	39
3.3.4	Dashboard, board și task-uri	40
3.3.5	Utilizarea funcțiilor AI	42
3.3.6	Rapoarte și documentație	43
3.3.7	Contul utilizatorului și administrarea globală	45
3.3.8	Manual scurt de utilizare	46
3.3.9	Exemplu complet de utilizare	47
4	Discutarea rezultatelor obținute	48
4.1	Testare și rezultate	48
4.1.1	Strategia de testare	48
4.1.2	Mediul de testare	48
4.1.3	Testarea API	48
4.1.4	Testarea UI desktop și mobilă	49
4.1.5	Testarea funcționalităților AI	49
4.1.6	Raportul Playwright	49
4.1.7	Interpretarea rezultatelor	50
4.1.8	Limitările testării	51
4.2	Evaluarea soluției și discutarea rezultatelor	51
4.2.1	Raportarea la obiectivele lucrării	51
4.2.2	Semnificația rezultatelor de testare	51
4.2.3	Raportarea la soluțiile existente	52
4.2.4	Valoarea practică și limitele soluției	52
5	Concluzii și direcții viitoare	53
5.1	Sinteza rezultatelor	53
5.2	Ce a funcționat și dificultăți întâmpinate	53
5.3	Probleme rămase deschise	53
5.4	Direcții viitoare	54
	Bibliografie	54

Capitolul 1

Introducere

1.1 Contextul și problema abordată

Managementul proiectelor software reprezintă o componentă importantă a ciclului de viață al dezvoltării software. O echipă care construiește un produs informatic trebuie să transforme cerințele în activități concrete, să urmărească progresul, să colaboreze, să controleze schimbările și să păstreze o evidență a deciziilor luate. În lipsa unei platforme centralizate, aceste informații ajung frecvent să fie împărțite între conversații, documente, repository-uri, foi de calcul și ședințe.

Metodologiile agile sunt relevante deoarece proiectele software se schimbă în timpul dezvoltării. Scrum organizează activitatea în sprinturi și definește roluri, evenimente și artefacte care susțin inspecția și adaptarea regulată [SS20]. Kanban pune accent pe vizualizarea fluxului, limitarea lucrului în desfășurare și îmbunătățirea continuă [VY20]. Scrumban combină planificarea iterativă cu un flux mai flexibil, apropiat de Kanban.

O problemă practică este trasabilitatea. Un task poate porni de la o cerință, poate fi rafinat și estimat, discutat în comentarii, mutat pe board, implementat într-un branch, revizuit printr-un pull request și inclus într-un raport. Dacă aceste informații nu sunt legate, echipa pierde contextul, iar justificarea unei schimbări devine dificilă.

1.2 Motivație

Din experiența dezvoltării aplicației, administrarea unui proiect software devine dificilă atunci când echipa folosește produse separate pentru planificare, task-uri, documentație, rapoarte și integrarea cu repository-ul de cod. Aplicațiile simple sunt ușor de folosit, dar pot avea roluri, audit sau raportare limitate. Produsele complexe acoperă mai multe nevoi, însă configurarea lor poate deveni dificilă pentru echipe mici sau pentru proiecte universitare.

O a doua motivație este flexibilitatea metodologică. O echipă poate începe cu Scrum și poate constata ulterior că are nevoie de flux continuu; alta poate prefera Scrumban. Dacă metodologia este doar un câmp text, aplicația nu sprijină această schimbare. În SDLC Hub, metodologia influențează paginile, câmpurile, regulile, rapoartele și permisiunile.

Inteligența artificială oferă o a treia motivație. Cercetările recente discută utilizarea AI pentru analiză, sumarizare, estimare și sprijinirea activităților agile [MUAZ⁺24]. Modelele de limbaj pot sprijini și ingineria cerințelor, de exemplu prin transformarea descrierilor informale în specificații mai clare [ZDA⁺25]. În această lucrare, AI-ul este integrat în fluxuri concrete și nu este tratat ca un chatbot separat.

1.3 Descrierea informală a soluției

SDLC Hub este o platformă web full-stack pentru organizarea și monitorizarea proiectelor software. Utilizatorul își creează un cont, se autentifică, creează sau accesează un proiect, alege metodologia, invită membri, definește roluri și gestionează activitatea. Project Wizard colectează identitatea proiectului, include un interviu asistat de AI pentru recomandarea metodologiei, pregătește rolurile și oferă pași pentru configurarea integrării DevOps.

După crearea proiectului, utilizatorul accesează un workspace care conține dashboard, board, backlog, task-uri, calendar, documentație, activitate, rapoarte, workload, notificări, setări și integrare GitHub. Administratorul global are o consolă separată pentru supravegherea utilizatorilor, proiectelor, tichetelor de suport, erorilor HTTP și utilizării funcțiilor AI.

1.4 Scop și obiective

Scopul lucrării este proiectarea, implementarea și validarea unei platforme web care centralizează activitățile importante ale unui proiect software și oferă asistență bazată pe inteligență artificială fără a elimina controlul utilizatorului. Soluția urmărește să reducă fragmentarea care apare atunci când planificarea, task-urile, documentația, rapoartele și activitatea din repository sunt gestionate în produse diferite. Din această perspectivă, obiectivul nu este realizarea unei simple liste de sarcini, ci construirea unui spațiu de lucru în care datele introduse într-o etapă a proiectului pot fi reutilizate în etapele următoare.

Un prim obiectiv este susținerea proiectelor Scrum, Kanban și Scrumban prin reguli care influențează efectiv comportamentul aplicației. Metodologia aleasă determină disponibilitatea backlog-ului, a sprinturilor, a story points, a limitelor WIP și a anumitor rapoarte. Un al doilea obiectiv este gestionarea coerentă a proiectelor, rolurilor, permisiunilor, task-urilor, sprinturilor și echipelor, astfel încât fiecare operație importantă să fie verificată atât în interfață, cât și în backend.

Lucrarea urmărește, de asemenea, păstrarea trasabilității prin comentarii, jurnal de audit, notificări, documentație internă și integrare GitHub. Componenta AI este proiectată local-first, prin Ollama și modelul `qwen2.5-coder:7b`, iar rezultatele sale sunt prezentate ca sugestii editabile. Obiectivele finale sunt furnizarea de actualizări realtime, rapoarte și instrumente de administrare globală, precum și validarea sistemului prin teste API, funcționale, desktop, mobile și responsive.

1.5 Contribuțiile autorului și rezultate

Contribuția principală este realizarea platformei SDLC Hub ca sistem integrat. Metodologia este tratată ca o regulă activă: ea determină disponibilitatea backlog-ului, sprinturilor, story points, limitelor WIP și anumitor rapoarte. Tranziția dintre metodologii folosește preview, avertismente, blocaje și audit, evitând modificările netransparente.

A doua contribuție este integrarea funcțiilor AI în activități operaționale: recomandarea metodologiei și rolurilor, rafinarea specificațiilor, estimarea, analiza riscului, echilibrarea încărcării, release notes și documentația automată. Implementarea curentă folosește Ollama local și păstrează controlul uman asupra rezultatului.

Rezultatul este o aplicație care include autentificare, autorizare, board, backlog, sprinturi, calendar, notificări, wiki, rapoarte, export PDF, GitHub Webhooks, Web-Socket și Global Admin. Raportul Playwright disponibil pentru rularea din 9 iulie 2026 înregistrează 151 de teste, dintre care 150 trecute, niciun test eșuat și un test omis; aceste rezultate sunt analizate în Secțiunea 4.1.

1.6 Structura lucrării

Organizarea lucrării transpune cele cinci părți recomandate pentru o lucrare preponderent practică în cinci capitole principale. Capitolul întâi introduce problema, motivația, scopul, obiectivele și contribuțiile autorului. Capitolul al doilea prezintă problematica managementului proiectelor software, fundamentele Scrum, Kanban și Scrumban, utilizarea inteligenței artificiale în dezvoltarea software și soluțiile existente relevante.

Capitolul al treilea detaliază contribuția autorului și reunește analiza și proiectarea sistemului, implementarea aplicației, justificarea tehnologiilor, manualul de implementare, descrierea interfeței și manualul de utilizare. Capitolul al patrulea discută rezultatele obținute prin testare și prin raportarea soluției la obiectivele stabilite și la produsele analizate. Capitolul al cincilea sintetizează rezultatele, dificultățile, limitele și direcțiile viitoare. Bibliografia este prezentată separat, după cele cinci capitole, și cuprinde sursele citate în text.

Capitolul 2

Prezentarea problematicii abordate

2.1 Managementul proiectelor software

Managementul unui proiect software presupune coordonarea cerințelor, planificării, execuției, colaborării și raportării. Spre deosebire de un proces liniar, dezvoltarea software conține schimbări de prioritate, dependențe tehnice, revizii și incertitudine. O platformă utilă trebuie să permită atât vizualizarea stării curente, cât și păstrarea contextului deciziilor.

Problema este descompusă în mai multe subprobleme: organizarea unităților de lucru, controlul accesului, adaptarea la metodologie, trasabilitatea, comunicarea în timp real, integrarea activității din repository, raportarea și asistarea deciziilor. SDLC Hub abordează aceste subprobleme prin module conectate, nu prin pagini independente.

2.2 Scrum, Kanban și Scrumban

Scrum folosește iterații cu durată delimitată și separă product backlog-ul de sprint backlog. Ghidul Scrum definește responsabilități, evenimente și artefacte pentru inspectarea și adaptarea muncii [SS20]. În SDLC Hub, această abordare este reflectată prin backlog, sprint planning, sprint activ, story points și release notes.

Kanban urmărește fluxul continuu, vizualizarea muncii și limitarea WIP [VY20]. În aplicație, proiectele Kanban nu depind de sprinturi, iar board-ul este principala reprezentare a fluxului. Scrumban păstrează elemente de planificare și estimare, dar permite un flux mai flexibil. Analizele comparative arată că efectele Scrum și Kanban depind de contextul proiectului și al echipei [LGJO17]; prin urmare, aplicația nu impune o metodologie unică.

2.3 Inteligența artificială în dezvoltarea software

AI-ul poate susține activități repetitive sau care necesită sintetizarea unui volum mare de context. În proiectele software, exemplele relevante sunt rafinarea cerințelor, estimarea efortului, identificarea riscurilor și generarea documentației. Cercetarea

privind estimarea software subliniază importanța evaluării sistematice a tehnicilor folosite pentru sprijinirea planificării [MKA⁺21].

În SDLC Hub, funcțiile AI nu execută automat decizii cu impact asupra proiectului. Rezultatul este prezentat ca sugestie editabilă. Acest principiu este important deoarece un model poate produce răspunsuri incomplete, iar datele locale pot fi insuficiente. Folosirea Ollama permite rularea locală a modelului și reduce dependența de un serviciu cloud [Oll26].

2.4 Jira Software

Jira este o platformă matură pentru planificare, workflow-uri, board-uri, automatizări și raportare [Atl26a]. În plus, versiunile actuale oferă funcții AI pentru generarea și transformarea conținutului, căutare în limbaj natural și sumarizarea comentariilor [Atl26b]. Prin urmare, diferența față de SDLC Hub nu este simpla existență a AI-ului, ci modul de integrare, modelul de rulare și dimensiunea soluției.

Figura 2.1 ilustrează o interfață Jira folosită ca reper pentru analiza comparativă.

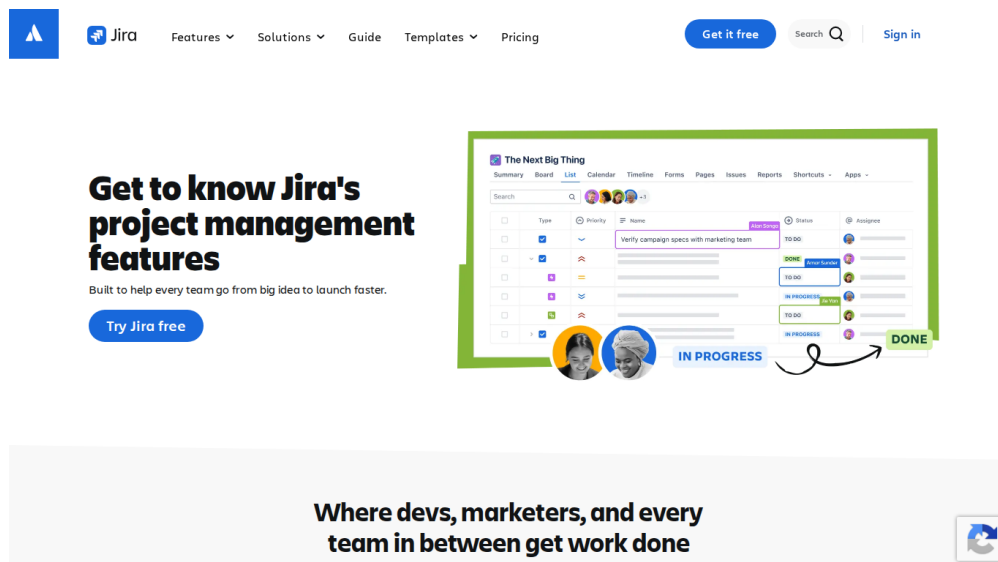


Figura 2.1: Interfață Jira Software, conform materialului de referință utilizat în lucrare

Avantajele Jira sunt configurabilitatea, ecosistemul extins și raportarea matură. Dezavantajele pentru contextul acestei lucrări sunt costul de configurare și complexitatea pentru echipe mici. SDLC Hub urmărește un set mai restrâns de fluxuri, controlate de metodologia proiectului.

2.5 Trello

Trello organizează munca prin board-uri, liste și carduri, oferind o interacțiune vizuală ușor de înțeles [Atl26c]. Platforma include automatizări și funcții AI pentru transformarea sau rezumarea conținutului [Atl26d]. Figura 2.2 prezintă pagina de ghid folosită pentru documentare.

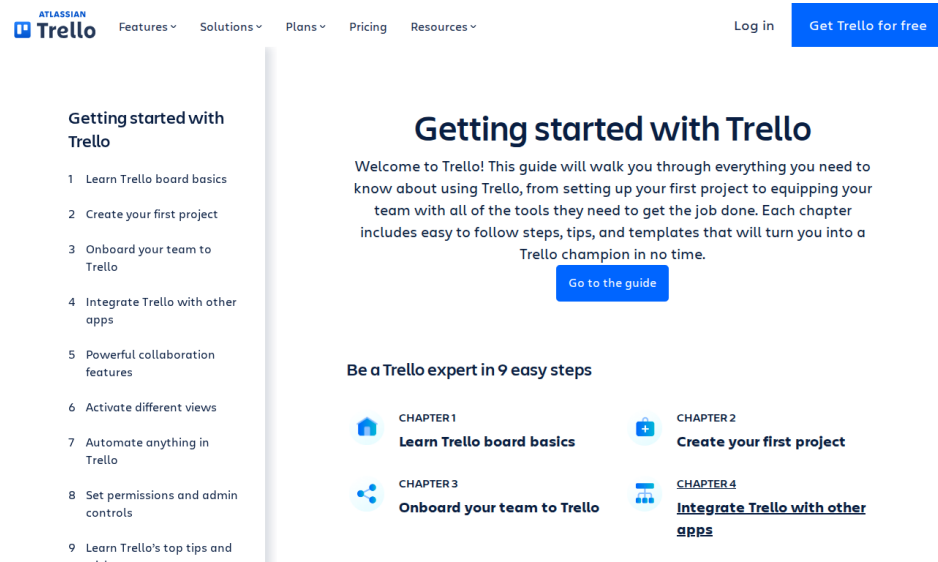


Figura 2.2: Pagina Trello Guide utilizată pentru analiza aplicației

Punctul forte al Trello este simplitatea. Pentru un proces software complex, rolurile granulare, sprinturile, auditul și raportarea pot necesita extensii sau alte produse. SDLC Hub păstrează modelul vizual al board-ului, dar îl conectează la cerințe, sprinturi, documentație și audit.

2.6 GitHub Projects

GitHub Projects apropie planificarea de issues, pull request-uri și repository-uri [Git26a]. GitHub oferă și funcții Copilot pentru înțelegerea și crearea issues, iar unele funcții sunt disponibile în regim preview sau în funcție de plan [Git26b]. Figura 2.3 prezintă documentația Projects utilizată în comparație.

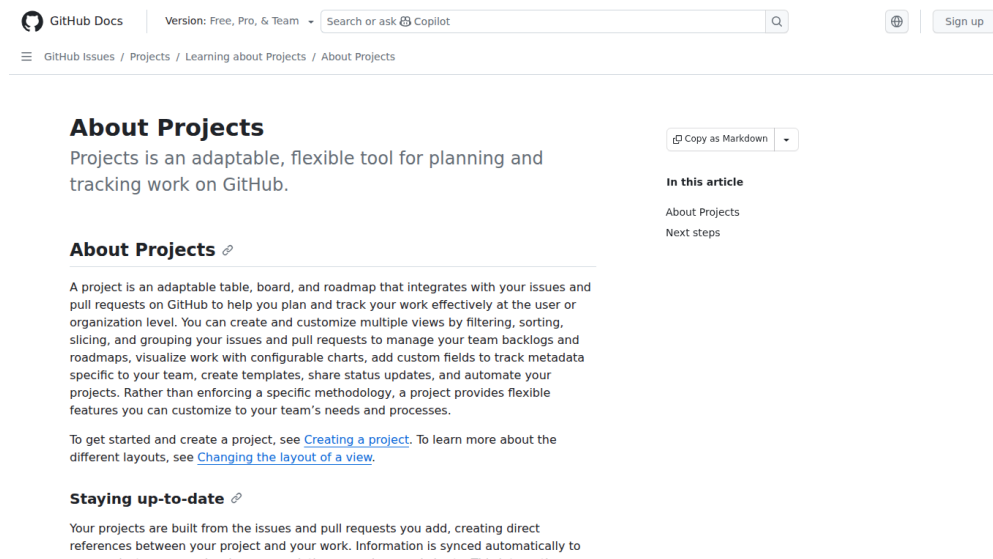


Figura 2.3: Documentația GitHub Projects utilizată pentru analiza aplicației

Avantajul principal este legătura directă cu activitatea din cod. În schimb, administrarea metodologiei, rolurile de proiect și consola globală nu sunt obiectivul central al produsului. SDLC Hub folosește GitHub ca integrare externă și păstrează managementul în propriul model de date.

2.7 Comparație obiectivă

Tabelul 2.1 sintetizează criteriile relevante pentru această lucrare. Comparația nu urmărește să prezinte SDLC Hub ca înlocuitor universal, ci să evidențieze scopul prototipului.

Criteriu	Jira	Trello	Git Hub Projects	SDLC Hub
Orientare principală Scrum/Kanban	Management configurabil Suport matur	Organizare vizuală Board flexibil	Planificare lângă cod Configurabil prin views	Flux SDLC integrat Reguli active Scrum/Kanban/Scrumban
AI	Rovo AI	Atlassian AI și extensii	Copilot	Ollama local-first și fallback
Trasabilitate cod-task	Prin integrări	Prin extensii	Nativă în ecosistem	Prin webhook și cheie de task
Administrare globală	Matură, enterprise	Simplificată	Organizație și repository	Consolă proprie de prototip
Complexitate inițială	Ridicată	Redusă	Medie pentru echipe tehnice	Medie, orientată spre demonstrație

Tabela 2.1: Comparație între aplicațiile analizate și SDLC Hub

Contribuția specifică SDLC Hub este combinația dintre tranzițiile metodologice controlate, funcțiile AI locale, auditul operațional și integrarea mai multor activități într-un proiect academic. Limitarea principală este maturitatea mai redusă față de produsele comerciale și necesitatea validării serviciilor externe în mediul de demonstrație.

Capitolul 3

Detalierea contribuției autorului

3.1 Analiza și proiectarea aplicației

3.1.1 Actori și roluri

Modelarea actorilor pornește de la separarea dintre accesul public, activitatea desfășurată într-un proiect și administrarea întregii platforme. Vizitatorul neautentificat poate consulta pagina publică, poate iniția înregistrarea și poate accesa fluxurile de autentificare sau recuperare a parolei. După autentificare, utilizatorul își gestionează profilul, sesiunile și proiectele disponibile, însă accesul la datele unui proiect depinde de existența unei apartenențe valide și de rolul atribuit.

În interiorul unui proiect există membri tehnici, precum Developer sau QA, și roluri de coordonare, precum Project Manager, Product Owner, Scrum Master, Flow Manager sau Tech Lead. Rolurile nu sunt tratate exclusiv ca denumiri afișate în interfață. Ele sunt asociate cu permisiuni care stabilesc ce operații pot fi executate asupra task-urilor, sprinturilor, echipei, documentației, rapoartelor și setărilor. Project Owner și Project Admin au responsabilități suplimentare: pot gestiona invitațiile, rolurile, metodologia, integrarea DevOps și providerul AI al proiectului.

Global Admin reprezintă un actor separat de administratorul unui proiect. Acesta nu lucrează în mod obișnuit cu board-ul unei echipe, ci supraveghează utilizatorii, proiectele, tichetele de suport, erorile HTTP și consumul funcțiilor AI la nivelul întregii platforme. În model apar și sisteme externe: serverul SMTP pentru e-mail, Ollama pentru inferența locală și GitHub pentru evenimentele DevOps. Separarea acestor actori clarifică limitele de încredere și justifică verificarea permisiunilor în backend, nu doar ascunderea unor controale în interfață.

Rolurile sunt implementate la nivel de proiect, ceea ce permite aceluiași utilizator să aibă responsabilități diferite în proiecte diferite. În proiectele Kanban, operațiile specifice sprinturilor sunt dezactivate, iar proiectele arhivate permit în principal consultarea datelor. Aceste reguli evită situația în care interfața afișează acțiuni incompatibile cu starea sau metodologia proiectului.

3.1.2 Cerințe funcționale

Cerințele funcționale principale sunt sintetizate în Tabelul 3.1.

ID	Cerință	Descriere
CF1	Identitate și cont	Înregistrare, verificare e-mail, login, resetare parolă, profil, avatar și sesiuni.
CF2	Proiecte și membri	Creare proiect, invitații, roluri, permisiuni, echipe și ownership.
CF3	Metodologie	Scrum, Kanban, Scrumban și tranziții cu preview, blockers și audit.
CF4	Work engine	Task-uri, subtasks, comentarii, board, backlog, sprinturi și audit.
CF5	Colaborare	Notificări, calendar, disponibilitate, documentație și realtime.
CF6	AI	Advisor, roluri, Spec Refiner, estimator, risc, workload, release notes.
CF7	DevOps	Configurare GitHub, webhook, commit/PR linking și audit.
CF8	Rapoarte	Dashboard, metrice, grafice și export PDF.
CF9	Administrare	Support tickets, utilizatori, proiecte, erori HTTP și AI usage.

Tabela 3.1: Cerințe funcționale principale

3.1.3 Cerințe nefuncționale

Cerințele nefuncționale au fost formulate pornind de la riscurile unui sistem care gestionează date despre utilizatori, proiecte și activitate tehnică. Securitatea presupune stocarea parolelor sub formă hash, autentificare prin JWT, verificarea permisiunilor pentru fiecare operație sensibilă, criptarea cheilor providerilor AI configurați de proiect și validarea semnăturii webhook-urilor GitHub. Aceste măsuri nu transformă prototipul într-un produs enterprise, dar reduc riscurile elementare și demonstrează separarea responsabilităților.

Trasabilitatea este o cerință distinctă deoarece datele proiectului trebuie să explice nu numai starea curentă, ci și modul în care s-a ajuns la ea. Jurnalul de audit, comentariile, notificările și legăturile dintre task-uri, sprinturi, documentație și GitHub contribuie la păstrarea contextului. Utilizabilitatea urmărește o navigare consecventă, feedback vizual și adaptarea interfeței la ecrane desktop și mobile, fără a ascunde complexitatea necesară unui proiect software.

Modularitatea și mentenabilitatea sunt susținute prin separarea frontend-ului, backend-ului, serviciilor și persistenței. Migrațiile Alembic permit evoluția controlată a schemei, iar componentele și serviciile reutilizabile reduc duplicarea. Pentru continuitatea anumitor fluxuri AI, aplicația poate utiliza un fallback determinist atunci când providerul local nu răspunde. Configurarea prin variabile de mediu permite rularea componentelor în contexte locale diferite, fără introducerea secretelor în codul sursă.

Aplicația este un prototip funcțional dezvoltat pentru lucrarea de licență și pen-

tru demonstrarea unei arhitecturi complete. Nu este prezentată ca serviciu enterprise pregătit pentru expunere publică la scară mare. Limitele privind încărcarea concurentă, securitatea avansată și disponibilitatea serviciilor externe sunt discutate în Capitolul al patrulea.

3.1.4 Cazuri de utilizare

Figura 3.1 prezintă cazurile de utilizare principale și relația dintre actori, module și sisteme externe.

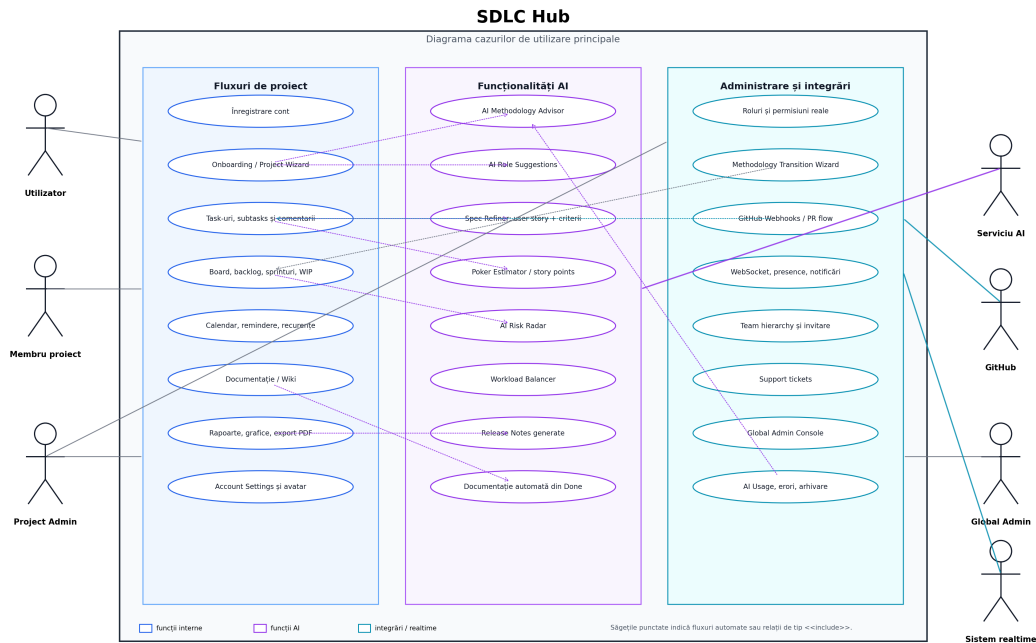


Figura 3.1: Diagrama cazurilor de utilizare principale ale platformei SDLC Hub

3.1.4.1 Detalii ale cazurilor de utilizare

În continuare sunt prezentate cazurile de utilizare considerate reprezentative pentru aplicație. Spre deosebire de o aplicație simplă de tip listă de task-uri, SDLC Hub are fluxuri care combină managementul proiectului, metodologia activă, permisiunile, AI-ul, raportarea și integrarea DevOps. Partea de autentificare și delogare nu este detaliată în această secțiune, deoarece lucrarea se concentrează pe fluxurile care diferențiază platforma: înregistrarea, crearea proiectului, managementul task-urilor, raportarea și funcționalitățile asistate de AI.

Înregistrare:

Actor: utilizator nou.

Descriere: utilizatorul își creează un cont în SDLC Hub completând formularul de înregistrare. Formularul conține numele complet, adresa de e-mail, parola și confirmarea parolei. După trimiterea datelor, backend-ul validează informațiile, verifică unicitatea e-mailului și creează contul.

Precondiții: utilizatorul are acces la pagina publică de înregistrare.

Postcondiții: contul este creat, iar utilizatorul poate continua fluxul de verificare și accesare a spațiului de lucru. Succesiunea interacțiunilor este prezentată în Figura 3.2.

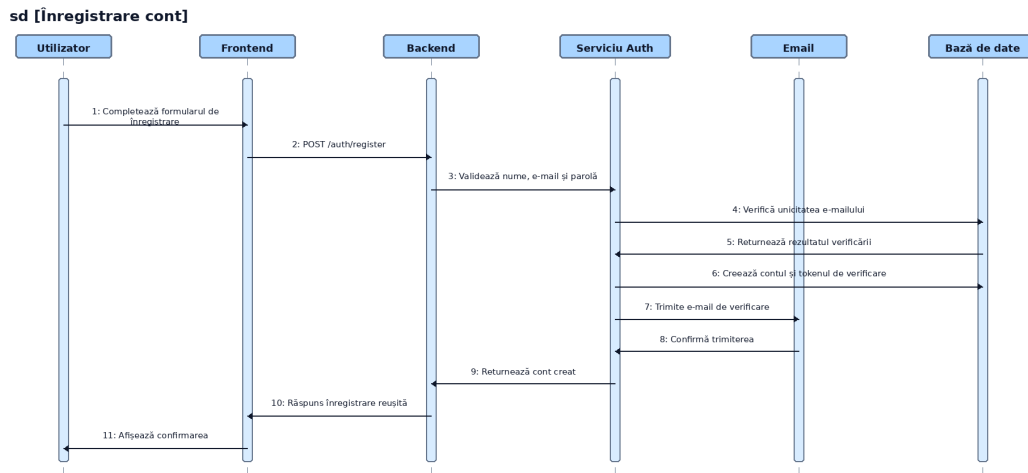


Figura 3.2: Diagrama de secvență pentru înregistrare

Creare proiect cu recomandare metodologie:

Actor: utilizator înregistrat.

Descriere: utilizatorul creează un proiect nou prin Project Wizard. Acesta completează identitatea proiectului, răspunde la întrebările interviului AI, primește o recomandare de metodologie și configurează rolurile inițiale.

Precondiții: utilizatorul are un cont valid și poate accesa Project Wizard.

Postcondiții: proiectul este creat, metodologia este salvată, rolurile sunt generate, iar utilizatorul devine administrator al proiectului. Fluxul complet este prezentat în Figura 3.3.

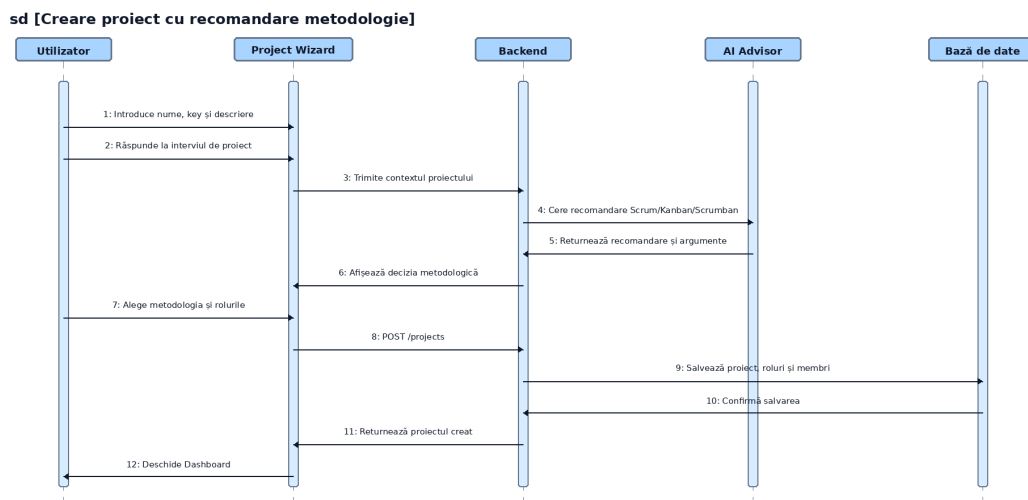


Figura 3.3: Diagrama de secvență pentru crearea proiectului

Gestionare task:

Actor: membru al proiectului.

Descriere: utilizatorul poate crea, edita sau muta task-uri pe board. În funcție de permisiuni, acesta poate schimba statusul, prioritatea, persoana asignată, echipa, due date-ul, story points și descrierea. Pentru Scrum și Scrumban se poate lucra cu sprinturi, iar pentru Kanban accentul cade pe flux continuu și limite WIP.

Precondiții: proiectul există, utilizatorul este membru și are permisiunile necesare.

Postcondiții: task-ul este actualizat, audit log-ul este completat, iar paginile conectate primesc actualizare realtime. Interacțiunea dintre componente este redată în Figura 3.4.

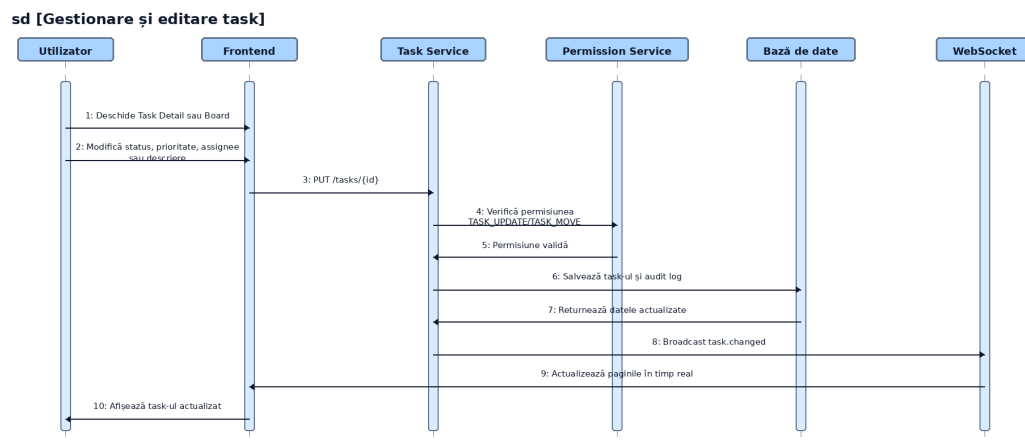


Figura 3.4: Diagrama de secvență pentru gestionarea task-ului

Export raport PDF:

Actor: utilizator cu acces la rapoarte.

Descriere: utilizatorul deschide pagina Reports și exportă un raport PDF cu starea proiectului. Raportul folosește date din task-uri, sprinturi, audit log și metrice de livrare.

Precondiții: proiectul conține date raportabile.

Postcondiții: raportul PDF este generat și poate fi descărcat. Secvența de generare și returnare a fișierului este prezentată în Figura 3.5.

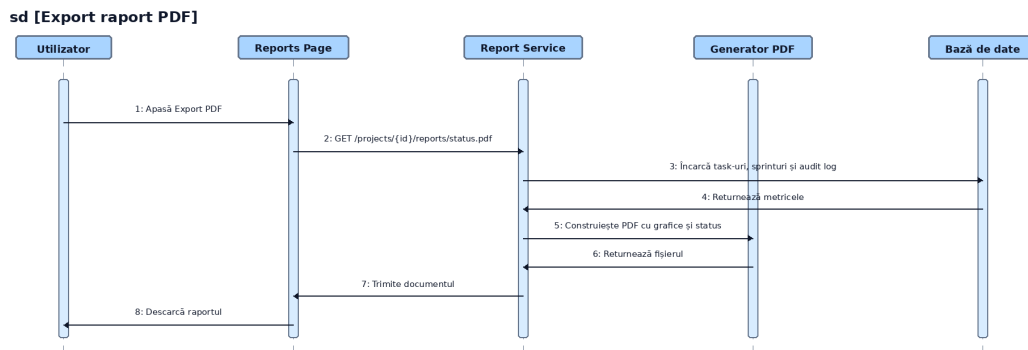


Figura 3.5: Diagrama de secvență pentru exportul raportului PDF

3.1.4.2 Funcționalități asistate de inteligența artificială

Funcționalitățile AI reprezintă una dintre contribuțiile importante ale aplicației. Ele au fost separate în mai multe fluxuri, deoarece fiecare rezolvă o problemă diferită din managementul proiectelor software. AI-ul nu este folosit ca un mecanism care ia decizii automat în locul echipei. El produce sugestii structurate, explicate și editabile, iar utilizatorul decide dacă le aplică.

AI Methodology Advisor. În Project Wizard, utilizatorul răspunde la un set de întrebări despre tipul proiectului, ritmul de lucru, nivelul de incertitudine, experiența echipei și nevoia de planificare. Backend-ul trimite acest context către serviciul AI, iar rezultatul este o recomandare între Scrum, Kanban și Scrumban. Recomandarea conține și explicații, astfel încât utilizatorul înțelege de ce o metodologie este mai potrivită pentru proiectul respectiv.

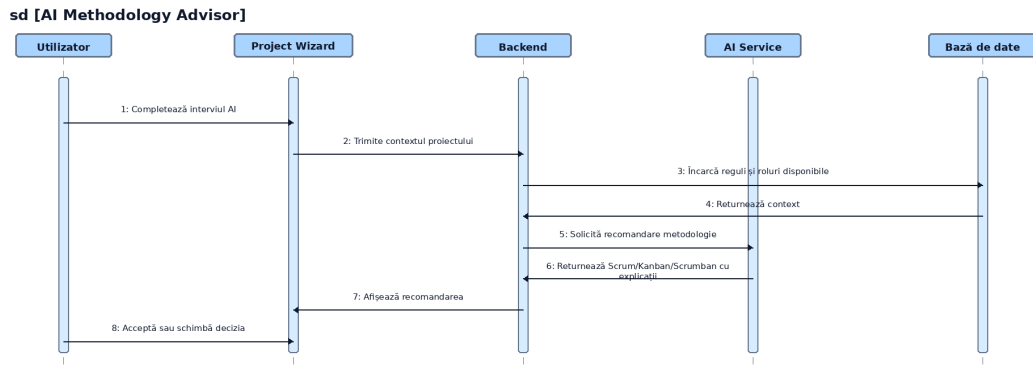


Figura 3.6: Diagrama de secvență pentru AI Methodology Advisor

Spec Refiner. La crearea unui task sau în pagina de detaliu a task-ului, utilizatorul poate porni de la o descriere scurtă sau neclară. Spec Refiner transformă această descriere într-o cerință mai structurată: user story, acceptance criteria, note tehnice și subtasks propuse. Această funcție este utilă deoarece multe cerințe ajung inițial în proiect sub formă de idei incomplete. AI-ul ajută la clarificare, dar utilizatorul poate modifica rezultatul înainte de salvare.

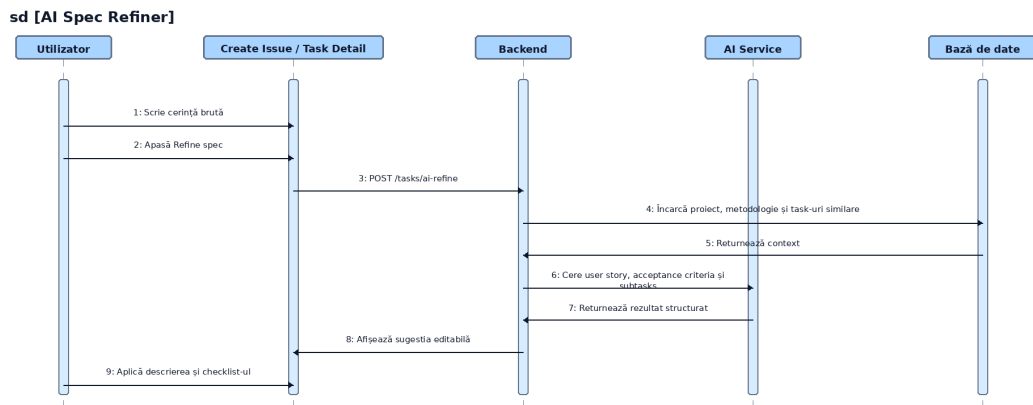


Figura 3.7: Diagrama de secvență pentru Spec Refiner

Poker Estimator. Pentru proiectele Scrum și Scrumban, estimarea prin story points este importantă pentru planificare și pentru urmărirea velocity-ului. Poker Estimator analizează titlul, descrierea, acceptance criteria, subtasks și contextul proiectului, apoi propune o estimare. Rezultatul include story points, confidence și reasoning. Estimarea nu este aplicată forțat, ci este afișată ca sugestie pe care utilizatorul o poate accepta sau modifica.

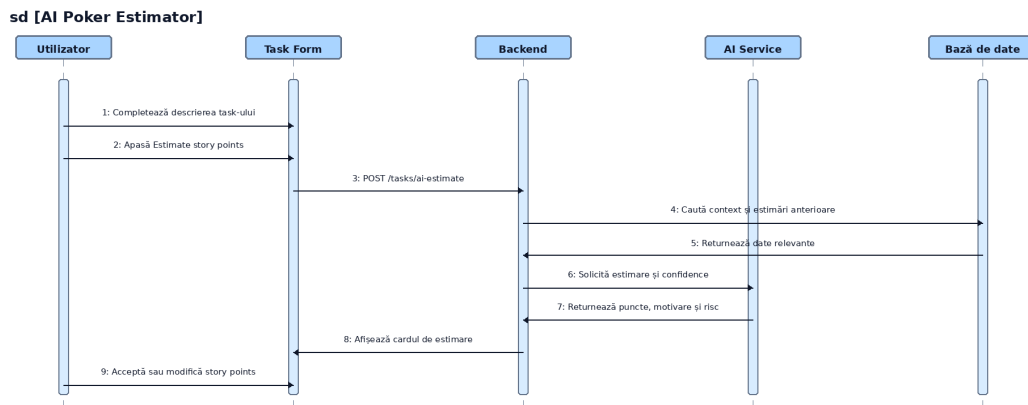


Figura 3.8: Diagrama de secvență pentru Poker Estimator

AI Risk Radar și Workload Balancer. AI Risk Radar combină semnale din task-uri, comentarii, due dates, audit log, prioritate și status. Sunt identificate task-uri întârziate, comentarii care indică blocaje, elemente critice neasignate sau task-uri care stau prea mult în Review/In Progress. Workload Balancer folosește semnale similare pentru a analiza capacitatea membrilor și pentru a propune redistribuiri. Sugestiile includ explicații, astfel încât echipa să vadă de ce un task ar trebui mutat sau reassignat.

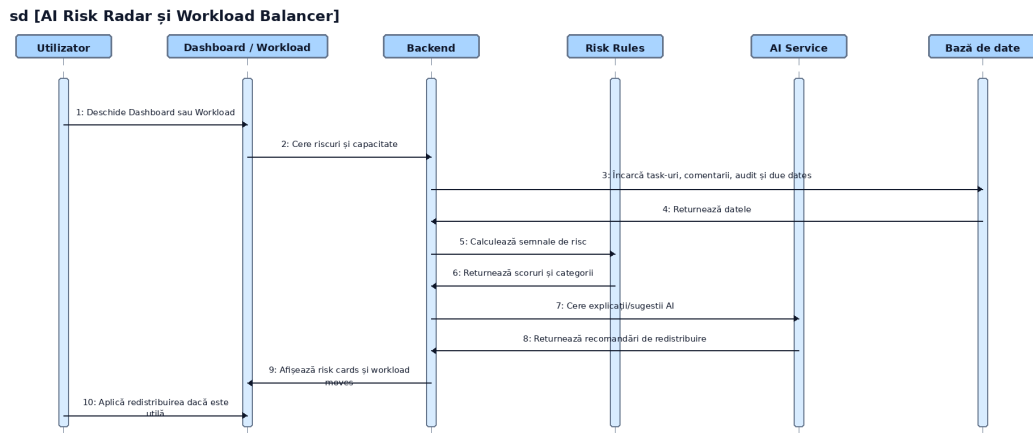


Figura 3.9: Diagrama de secvență pentru AI Risk Radar și Workload Balancer

Release Notes și documentație automată. După închiderea unui sprint sau finalizarea unor task-uri importante, aplicația poate genera release notes. AI-ul folosește task-urile Done, comentariile relevante și evenimentele GitHub pentru a crea un rezumat al livrării. În paralel, când un task ajunge în Done, aplicația poate genera sau actualiza o pagină de documentație. Astfel, cunoașterea tehnică nu rămâne blocată doar în comentarii sau pull request-uri.

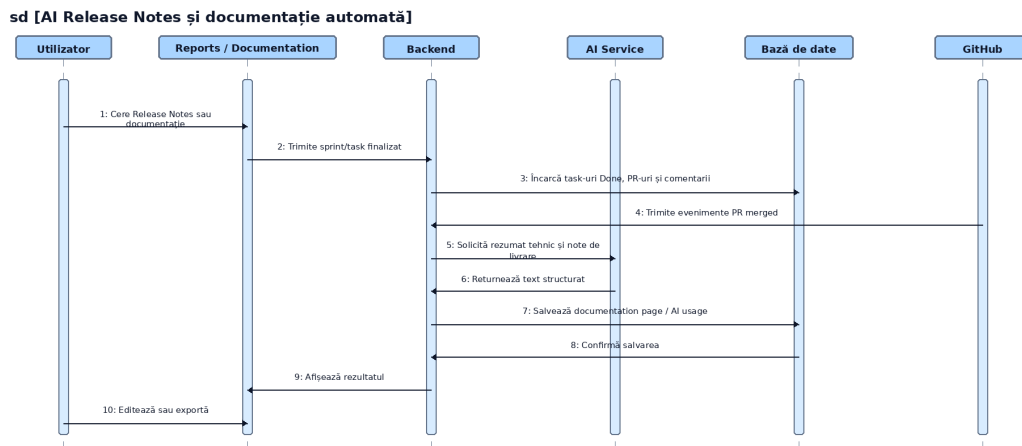


Figura 3.10: Diagrama de secvență pentru Release Notes și documentație automată

AI Usage în Global Admin. Pentru administrarea platformei, cererile AI sunt înregistrate în `ai_usage_logs`. Consola Global Admin poate afișa feature-ul folosit, providerul, statusul, utilizatorul, proiectul și data cererii. Această funcție este utilă pentru monitorizare, audit și diagnosticare atunci când un provider AI nu răspunde corect sau se folosește fallback-ul local.

3.1.5 Arhitectura sistemului

Arhitectura pe trei niveluri separă prezentarea, logica aplicației și datele [IBM26]. În SDLC Hub, frontend-ul Next.js comunică prin REST și WebSocket cu backend-ul FastAPI, iar backend-ul persistă datele prin SQLAlchemy în PostgreSQL. Figura 3.11 prezintă această organizare.

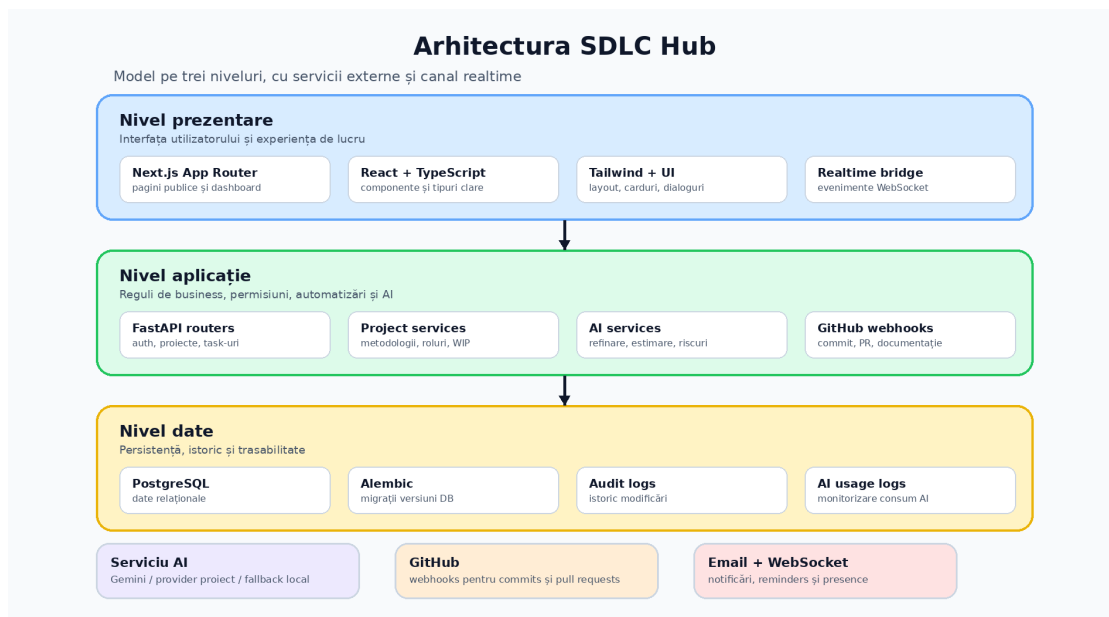


Figura 3.11: Arhitectura pe trei niveluri a platformei SDLC Hub

Figura 3.12 detaliază relația dintre modulele frontend și backend. Rutele publice, workspace-ul și consola Global Admin folosesc servicii de acces la API, iar backend-ul este împărțit în routere, servicii, modele, scheme și utilitare.

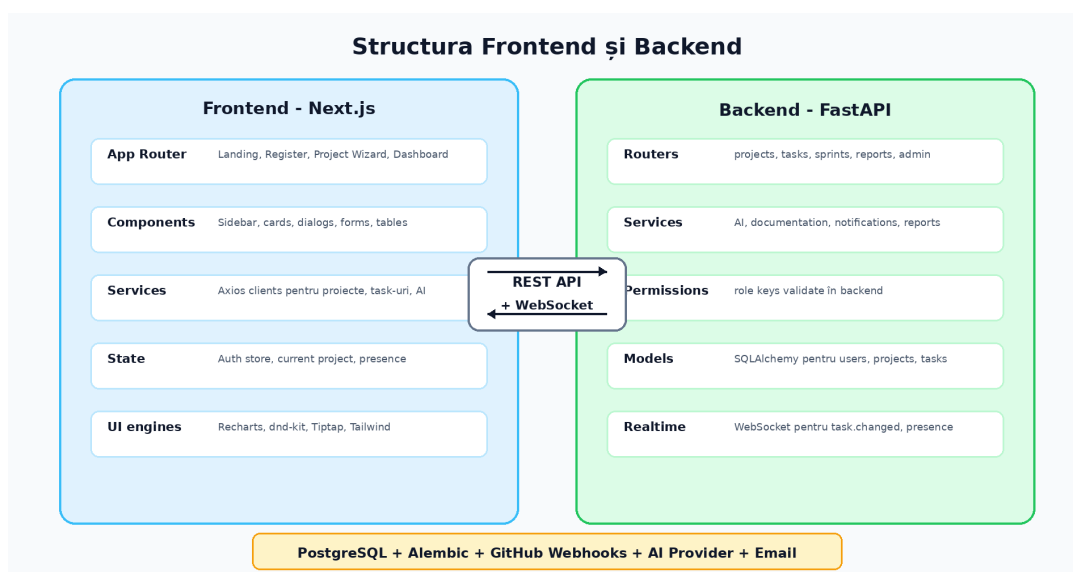


Figura 3.12: Structura principală a componentelor frontend și backend

3.1.6 Modelul datelor

PostgreSQL este utilizat pentru persistență [Pos26]. Entitățile principale sunt utilizatorul, proiectul, apartenența la proiect, rolul, task-ul, sprintul, subtask-ul, comentariul, auditul, evenimentul de calendar, notificarea, pagina de documentație, evenimentul GitHub, tichetul de suport și jurnalul utilizării AI.

Figura 3.13 prezintă modelul conceptual. Relațiile permit trasabilitatea: un task poate aparține unui proiect și unui sprint, poate avea comentarii, subtasks, audit, evenimente GitHub și documentație asociată.

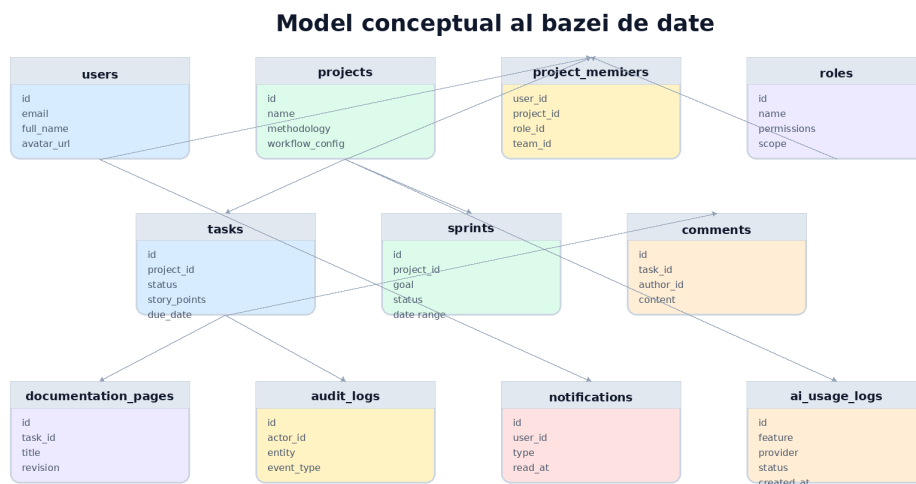


Figura 3.13: Modelul conceptual al datelor persistente

3.1.7 API și interfețe externe

API-ul este organizat pe domenii funcționale, astfel încât autentificarea, proiectele, task-urile, sprinturile, calendarul, echipele, documentația, GitHub, notificările, activitatea și administrarea globală să poată evolua separat. FastAPI oferă rutare, validarea datelor și documentare automată [Fas26], iar schemele de intrare și ieșire sunt separate de modelele ORM. Această separare previne expunerea directă a structurii interne a bazei de date și face mai clar contractul dintre frontend și backend.

Interfața SMTP este folosită pentru verificarea adresei de e-mail și resetarea parolei. Ollama este accesat prin API-ul său local pentru verificarea modelelor disponibile și pentru generarea răspunsurilor AI. GitHub Webhooks transmite notificări HTTP pentru evenimentele configurate în repository [Git26d]. În SDLC Hub sunt procesate evenimente de tip `push` și `pull_request`, iar backend-ul validează semnătura înainte de a extrage cheile task-urilor și de a actualiza istoricul DevOps. Comunicarea WebSocket păstrează o conexiune bidirecțională între brow-

ser și server [MDN26] și este utilizată pentru evenimente precum `task.changed`, `project.changed`, `github.changed` și `notification.changed`.

Aceste interfețe au roluri diferite. REST este canalul prin care se solicită și se persistă o modificare, WebSocket anunță ceilalți clienți că datele s-au schimbat, SMTP livrează mesaje în afara aplicației, GitHub furnizează activitate tehnică, iar Ollama procesează solicitările AI. Delimitarea lor evită transformarea unui singur mecanism într-o dependență universală și permite tratarea separată a erorilor.

3.1.8 Fluxuri importante

Figura 3.14 sintetizează ciclul unui proiect: onboarding, creare proiect, planificare, execuție, review, documentare și raportare.

Fluxul de creare a proiectului

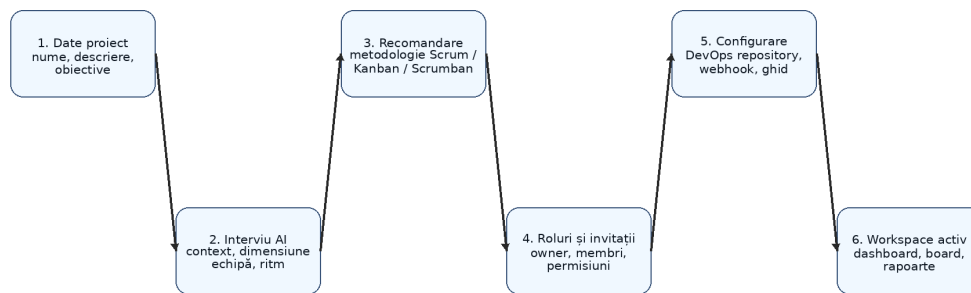


Figura 3.14: Fluxul principal al unui proiect în SDLC Hub

Fluxurile AI au fost modelate separat deoarece folosesc contexte și rezultate diferite. Advisor-ul metodologic este prezentat în Figura 3.6, Spec Refiner în Figura 3.7, estimatorul în Figura 3.8, analiza de risc și workload în Figura 3.9, iar release notes și documentația în Figura 3.10.

3.1.9 Exemplu de utilizare urmărit

Un exemplu reprezentativ începe cu Project Wizard. Owner-ul definește proiectul și confirmă metodologia recomandată. Echipa creează un task, îl rafinează și îl estimează, apoi îl planifică într-un sprint sau îl introduce în fluxul Kanban. Task-ul

este mutat pe board, iar commit-ul sau pull request-ul care conține cheia task-ului este asociat prin webhook. La final, task-ul ajunge în *Done*, poate genera documentație, apare în raport și contribuie la metrice. Acest exemplu arată legătura dintre proiectare, implementare, DevOps și raportare.

3.2 Implementarea aplicației

3.2.1 Tehnologii și justificarea alegerilor

Backend-ul este realizat în Python cu FastAPI [Fas26], SQLAlchemy [SQL26b], Alembic [SQL26a] și PostgreSQL [Pos26]. FastAPI a fost ales pentru structurarea API-urilor și validarea datelor, SQLAlchemy pentru maparea obiect-relațională, Alembic pentru evoluția controlată a schemei, iar PostgreSQL pentru relațiile și tranzacțiile necesare aplicației.

Frontend-ul folosește Next.js [Ver26], React [Met26], TypeScript și Tailwind CSS [Tai26]. Componentele UI sunt construite cu elemente Radix/shadcn-style [Rad26], starea client este gestionată prin Zustand [pmn26], editorul rich text folosește Tiptap [Tip26], board-ul folosește dnd-kit [dnd26], iar graficele folosesc Recharts [Rec26]. Playwright este utilizat pentru automatizarea testelor [Mic26].

3.2.2 Organizarea backend-ului

Punctul de intrare `backend/main.py` inițializează aplicația, CORS, fișierele statice, middleware-ul de erori, routerele și endpoint-ul WebSocket. `backend/config.py` centralizează setările de mediu. Modelele sunt în `backend/models`, schemele în `backend/schemas`, rutele în `backend/routers`, iar logica reutilizabilă în `backend/services` și `backend/utils`.

Această separare reduce cuplarea dintre transportul HTTP, logica de business și persistență. De exemplu, ruta unui task validează utilizatorul și permisiunea, serviciile aplică regulile, iar modelul ORM persistă rezultatul și auditul.

3.2.3 Organizarea frontend-ului

Next.js App Router separă paginile publice, fluxurile de autentificare, workspace-ul și consola Global Admin. Directorul `frontend/components` conține componente reutilizabile, `frontend/services` centralizează cererile HTTP, `frontend/store` păstrează starea globală, iar `frontend/hooks` conține logica reutilizabilă pentru evenimente și permisiuni.

TypeScript contribuie la consistența obiectelor transferate între API și UI. Componentele de task, board, dialoguri și tabele sunt separate pentru a evita concentrarea întregii logici într-o singură pagină.

3.2.4 Baza de date și migrațiile

Modelele SQLAlchemy descriu entitățile persistente și relațiile lor. Alembic păstrează istoricul modificărilor de schemă și permite aplicarea aceleiași versiuni pe un mediu nou. Operațiile care modifică mai multe entități sunt executate în aceeași sesiune, astfel încât proiectul, apartenența, rolurile și invitațiile să nu rămână într-o stare parțială.

3.2.5 Autentificare, permisiuni și securitate

Autentificarea folosește JSON Web Token, standardizat prin RFC 7519 [JBS15]. Parolele sunt păstrate sub formă hash, iar utilizatorul poate vedea și revoca sesiuni active. Verificarea e-mailului și resetarea parolei depind de configurarea SMTP.

Autorizarea se face pe proiect și permisiune. Ascunderea unui buton în frontend este doar o măsură de utilizabilitate; backend-ul refuză operațiile nepermise. Acțiunile sensibile, precum ștergerea unui task, invalidarea estimării, arhivarea sau transferul ownership-ului, cer confirmări și produc audit.

Cheile pentru providerii AI custom sunt criptate server-side și nu sunt trimise înapoi în browser. Secretele SMTP, JWT, AI și webhook sunt citite din variabile de mediu, nu sunt introduse în cod sau în lucrare.

3.2.6 Managementul proiectelor și task-urilor

3.2.6.1 Crearea și configurarea proiectului

Crearea unui proiect este realizată prin Project Wizard deoarece o simplă pagină cu nume și descriere nu ar acoperi deciziile necesare inițializării unui spațiu de lucru. Wizard-ul colectează identitatea proiectului, cheia folosită în identificatorii task-urilor, descrierea, metodologia și configurația inițială a echipei. În același flux sunt pregătite rolurile și invitațiile, astfel încât proiectul să nu fie creat într-o stare incompletă, fără owner sau fără reguli de acces.

Alegerea metodologiei produce o configurație efectivă. Pentru Scrum sunt activate backlog-ul, sprinturile și estimarea în story points. Kanban pune accent pe flux continuu, coloane și limite WIP. Scrumban păstrează posibilitatea planificării, dar permite o execuție mai apropiată de fluxul continuu. Această diferențiere explică de ce metodologia nu este stocată doar pentru afișare: ea influențează regulile, paginile disponibile și rapoartele.

Schimbarea ulterioară a metodologiei este tratată ca operație cu impact asupra datelor existente. Înainte de confirmare, backend-ul produce un preview care poate indica sprinturi active, câmpuri incompatibile, task-uri care trebuie mutate sau funcții care devin indisponibile. Avertismentele și blocajele reduc riscul unei tranziții care ar invalida planificarea existentă, iar jurnalul de audit păstrează decizia.

3.2.6.2 Ciclul de viață al task-ului

Task-ul este unitatea centrală de lucru și primește o cheie de forma `PROJ-1`, derivată din cheia proiectului. Pe lângă titlu și descriere, sunt păstrate statusul, prioritatea, termenul, responsabilul, echipa, sprintul și estimarea. Descrierea rich text permite introducerea contextului tehnic și a criteriilor de acceptare, iar subtask-urile descompun o cerință în activități verificabile.

Board-ul și pagina Tasks prezintă aceleași entități în scopuri diferite. Board-ul este potrivit pentru urmărirea stării și pentru mutarea rapidă între coloane, în timp ce lista filtrabilă este necesară pentru căutare, sortare și analizarea unui volum mai mare de activități. Mutarea unui card nu reprezintă doar o schimbare vizuală; frontend-ul solicită backend-ului actualizarea statusului, backend-ul verifică permisiunile și regulile metodologiei, apoi persistă modificarea și emite un eveniment realtime.

Pagina de detaliu concentrează informațiile necesare executării unei activități. Aici sunt modificate proprietățile, sunt create subtask-uri, sunt adăugate comentarii și poate fi urmărit jurnalul de audit. Comentariile și mențiunile păstrează discuția lângă obiectul la care se referă. Indicatorul de tastare și actualizările realtime reduc riscul ca doi membri să lucreze pe informații depășite.

3.2.6.3 Guvernanță și trasabilitate

Operațiile distructive sau cu efect asupra raportării sunt controlate separat. Ștergerea unui task, invalidarea unei estimări, arhivarea proiectului și schimbarea ownership-ului necesită permisiuni și confirmări. Scopul nu este doar prevenirea unei apăsări accidentale, ci și păstrarea unei responsabilități clare pentru modificări care afectează istoricul proiectului.

Trasabilitatea este construită prin legarea acelui task de comentarii, subtask-uri, sprint, evenimente GitHub, audit și documentația rezultată după finalizare. În acest mod, o cerință poate fi urmărită de la formularea inițială până la implementare, review și raportare. Această legătură reprezintă unul dintre motivele principale pentru care SDLC Hub a fost dezvoltat ca platformă integrată și nu ca board izolat.

3.2.7 Funcționalități asistate de inteligența artificială

Componenta AI este local-first. Providerul implicit este Ollama cu modelul `qwen2.5-coder:7b` [Oll26], iar owner-ul poate utiliza configurația platformei sau un provider propriu de tip Ollama ori OpenAI-compatible. Cheile providerilor custom sunt criptate server-side. Figura 3.15 prezintă poziția serviciului AI în raport cu proiectele, task-urile, rapoartele și documentația.

Arhitectura funcționalităților AI

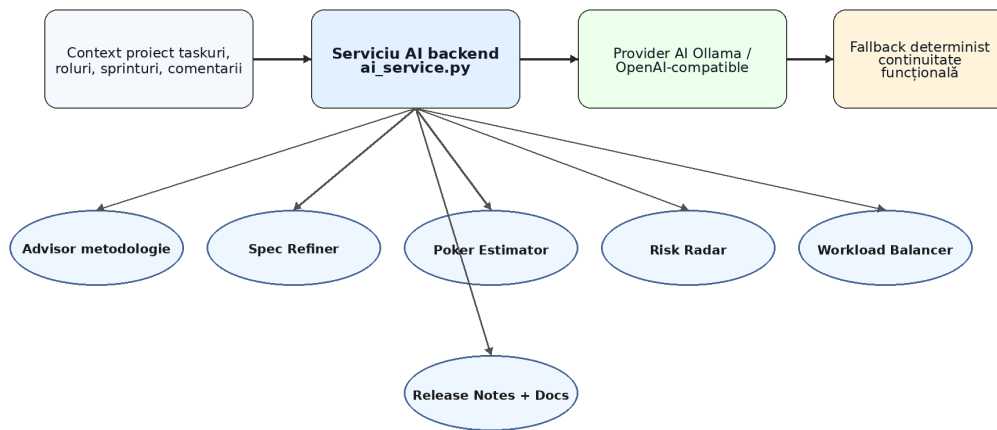


Figura 3.15: Arhitectura și fluxurile funcționalităților AI din SDLC Hub

3.2.7.1 Recomandarea metodologiei și a rolurilor

Smart Methodology Advisor este utilizat în etapa în care proiectul nu are încă o configurație stabilă. Utilizatorul răspunde la întrebări despre dimensiunea echipei, stabilitatea cerințelor, ritmul livrărilor, experiența membrilor și natura activității. Serviciul AI primește aceste informații și întoarce o metodologie recomandată, un nivel de încredere și o explicație. Recomandarea nu salvează automat proiectul și nu blochează alegerea manuală; rolul ei este să transforme răspunsurile într-un punct de plecare argumentat.

Sugestiile de roluri folosesc metodologia și descrierea proiectului pentru a propune o structură inițială a echipei. Această funcție răspunde unei probleme întâlnite la începutul proiectelor mici: rolurile sunt adesea definite prea general sau sunt copiate dintr-un model care nu corespunde modului real de lucru. Utilizatorul poate redenumi, elimina sau completa rolurile înainte de salvare, astfel încât AI-ul să rămână un instrument de configurare, nu o autoritate asupra structurii organizaționale.

3.2.7.2 Rafinarea specificațiilor și estimarea

Spec Refiner pornește de la o cerință informală, care poate conține doar intenția generală a utilizatorului. Serviciul generează o descriere mai structurată, criterii de acceptare și propuneri de subtask-uri. Funcția a fost introdusă deoarece task-urile foarte scurte transferă incertitudinea către dezvoltator și fac dificilă verificarea rezultatului. Textul generat este afișat înainte de aplicare și poate fi editat, iar utilizatorul decide ce informații sunt păstrate.

Poker Estimator analizează descrierea, complexitatea, dependențele și posibilele riscuri pentru a propune story points, încredere și factori care au influențat rezultatul. Estimarea AI nu înlocuiește discuția echipei și nu este folosită în Kanban atunci când proiectul nu utilizează story points. Valoarea principală constă în explicarea ipotezelor, astfel încât membrii să poată contesta sau ajusta propunerea în loc să accepte un număr fără justificare.

3.2.7.3 Analiza riscului și echilibrarea încărcării

AI Risk Radar analizează informațiile disponibile în task-uri și comentarii pentru a identifica semnale precum blocaje, dependențe neclarificate, termene apropiate sau formulări care indică incertitudine. Rezultatul este prezentat ca avertizare și nu schimbă automat prioritatea sau statusul. Această alegere este importantă deoarece riscul nu poate fi determinat complet doar din text, iar contextul echipei rămâne esențial.

Workload Balancer nu se limitează la numărarea task-urilor atribuite fiecărui membru. Sunt luate în considerare estimările, stările, termenele și disponibilitatea cunoscută, apoi sunt generate sugestii explicate de redistribuire. Managerul poate vedea motivul recomandării și decide dacă mutarea este realistă în raport cu specializarea membrilor. Funcția a fost concepută pentru a sprijini decizia, nu pentru a reasigna automat munca.

3.2.7.4 Release notes și documentație

Generarea release notes sintetizează task-urile finalizate într-un sprint sau într-o perioadă și produce un text care poate fi revizuit înainte de publicare. Documentația automată pornește de la un task finalizat și creează o propunere de pagină wiki, păstrând legătura cu activitatea sursă. Aceste două funcții reduc diferența dintre munca executată și artefactele care trebuie comunicate sau păstrate după livrare.

Dacă providerul AI nu răspunde, anumite funcții pot folosi fallback determinist pentru a păstra continuitatea demonstrației. Fallback-ul nu este prezentat ca rezultat al modelului, iar utilizatorul continuă să poată modifica sau respinge ieșirea. Prin această abordare, indisponibilitatea AI nu blochează managementul de bază al proiectului.

3.2.8 Comunicarea realtime prin WebSocket

Frontend-ul se conectează la `/ws` cu token JWT și context de proiect. Backend-ul verifică identitatea și apartenența, apoi transmite evenimente pentru task-uri, proiect, calendar, notificări, GitHub și activity. Clientul are mecanism de reconectare și reîncarcă doar zonele relevante.

Listarea 3.1 reproduce fragmentul real din `backend/main.py` care acceptă conexiunea numai după construirea contextului autentificat, trimite evenimentul inițial și răspunde la mesaje de tip `ping`. Închiderea conexiunii este tratată explicit.

```

1 @app.websocket("/ws")
2 async def websocket_endpoint(websocket: WebSocket):
3     token = websocket.query_params.get("token")
4     context = _get_websocket_context(token or "")
5
6     if not context:
7         await websocket.close(
8             code=status.WS_1008_POLICY_VIOLATION
9         )
10        return
11
12    await realtime_manager.connect(
13        websocket,
14        user_id=context["user_id"],
15        project_ids=context["project_ids"],
16    )
17    try:
18        await websocket.send_json({
19            "type": "connected",
20            "payload": {
21                "project_ids": list(context["project_ids"])
22            },
23        })
24        while True:
25            message = await websocket.receive_text()
26            if message == "ping":
27                await websocket.send_json({
28                    "type": "pong",
29                    "payload": {},
30                })
31        except WebSocketDisconnect:
32            await realtime_manager.disconnect(websocket)
33        except Exception:
34            await realtime_manager.disconnect(websocket)

```

Listarea 3.1: Endpoint-ul WebSocket autentificat din backend

Această implementare reduce nevoia de refresh manual și menține sincronizate board-ul, notificările și paginile de detaliu. WebSocket nu înlocuiește API-ul REST: modificarea este persistată prin API, iar evenimentul realtime anunță ceilalți clienți.

3.2.9 Integrarea GitHub Webhooks

Owner-ul configurează repository-ul și secretul. GitHub trimite payload-uri pentru evenimente precum `push` și `pull_request`; backend-ul validează semnătura HMAC înainte de procesare, conform recomandării GitHub de verificare a livrărilor [Git26c].

Cheile de task extrase din commit sau pull request permit asocierea cu auditul și pagina DevOps.

Listarea 3.2 este preluată din `backend/routers/github.py`. Semnătura așteptată este calculată cu SHA-256, iar comparația folosește `hmac.compare_digest`, evitând o comparație obișnuită dependentă de timp.

```
1 def _verify_signature(  
2     secret: str,  
3     body: bytes,  
4     signature: str | None,  
5 ) -> None:  
6     if not signature:  
7         raise HTTPException(  
8             status_code=status.HTTP_401_UNAUTHORIZED,  
9             detail="Missing GitHub signature.",  
10        )  
11  
12     expected = "sha256=" + hmac.new(  
13         secret.encode("utf-8"),  
14         body,  
15         hashlib.sha256,  
16     ).hexdigest()  
17  
18     if not hmac.compare_digest(expected, signature):  
19         raise HTTPException(  
20             status_code=status.HTTP_401_UNAUTHORIZED,  
21             detail="Invalid GitHub signature.",  
22        )
```

Listarea 3.2: Validarea semnăturii unei livrări GitHub

Listarea 3.3 arată extragerea cheilor de task din mesajele primite. Conversia la majuscule și mulțimea `seen` asigură un rezultat fără duplicate, în ordinea primei apariții.

```
1 TASK_KEY_RE = re.compile(r"\b[A-Z] [A-Z0-9]+\d+\b")  
2  
3 def _extract_task_keys(  
4     *values: str | None,  
5 ) -> list[str]:  
6     keys: list[str] = []  
7     seen: set[str] = set()  
8  
9     for value in values:  
10         if not value:  
11             continue  
12         for match in TASK_KEY_RE.findall(value.upper()):
```

```

13         if match not in seen:
14             keys.append(match)
15             seen.add(match)
16
17     return keys

```

Listarea 3.3: Extragerea cheilor de task din datele GitHub

În dezvoltare, endpoint-ul local poate fi expus prin ngrok. Funcționalitatea depinde de un repository și de un webhook configurate corect.

3.2.10 Calendar, notificări și activitate

Calendarul completează board-ul deoarece nu toate evenimentele unui proiect pot fi reprezentate ca task-uri. Ședințele, datele de disponibilitate, concediile și termenele comune au nevoie de o vedere temporală. Evenimentele pot include legături de întâlnire, iar informațiile despre disponibilitate sunt relevante pentru interpretarea încărcării echipei.

Notificările transmit utilizatorului schimbările care îl privesc fără a-l obliga să verifice permanent fiecare pagină. Ele sunt generate pentru evenimente precum invitații, atribuirii, mențiuni sau modificări relevante. Numărul de notificări necitite și actualizările WebSocket fac vizibilă schimbarea aproape imediat, în timp ce operația propriu-zisă rămâne salvată prin API.

Pagina Activity și jurnalele de audit răspund unei nevoi diferite de notificări. Notificarea atrage atenția asupra unui eveniment curent, în timp ce activitatea permite reconstruirea istoricului proiectului. Împreună, aceste module contribuie la transparență și la explicarea deciziilor.

3.2.11 Rapoarte, documentație și Global Admin

3.2.11.1 Dashboard și rapoarte

Dashboard-ul agregă informațiile care trebuie înțelese imediat după deschiderea proiectului: gradul de finalizare, munca activă, semnalele de întârziere, starea echipei și activitatea recentă. El nu înlocuiește paginile detaliate, ci reduce timpul necesar identificării unei probleme. Indicatorii sunt calculați din task-uri, sprinturi și termene, ceea ce înseamnă că valoarea lor depinde de actualizarea corectă a datelor operaționale.

Pagina Reports oferă o analiză mai amplă și adaptează metricele la metodologie. Pentru proiectele cu sprinturi sunt relevante velocity, burndown și progresul în story points. Pentru fluxul Kanban sunt urmărite lead time, cycle time, cumulative flow și WIP. Exportul PDF este generat în backend și produce un artefact care poate fi arhivat sau transmis stakeholderilor. Rapoartele nu sunt prezentate ca măsurători perfecte ale performanței individuale, ci ca instrumente pentru observarea tendințelor proiectului.

3.2.11.2 Documentația internă

Modulul Documentation funcționează ca wiki al proiectului și păstrează pagini cu revizii. El a fost introdus pentru a evita separarea completă dintre task-urile finalizate și cunoștințele rezultate din implementare. O activitate închisă poate deveni sursă pentru o pagină de documentație, iar legătura către task păstrează contextul tehnic și discuțiile care au dus la rezultat.

3.2.11.3 Administrarea globală

Global Admin este separat de workspace deoarece administrarea platformei are alte obiective decât coordonarea unui proiect. Consola oferă o vedere asupra utilizatorilor, proiectelor, tichetelor de suport, erorilor HTTP și utilizării funcțiilor AI. Separarea reduce riscul ca operațiile globale să fie confundate cu permisiunile unui Project Admin și permite urmărirea stării generale a sistemului.

Jurnalul erorilor HTTP ajută la identificarea rutelor și situațiilor care produc răspunsuri problematice, iar AI usage permite observarea frecvenței și duratei solicitărilor. Exporturile oferă o formă de analiză externă. Modulul nu este prezentat ca soluție completă de observabilitate, dar demonstrează necesitatea unei perspective operaționale asupra unei aplicații cu mai mulți utilizatori și proiecte.

3.2.12 Manual de implementare și instalare

3.2.12.1 Cerințe locale

Mediul documentat folosește Python 3.11 sau mai nou, Node.js 20 sau mai nou, PostgreSQL 16 și Ollama pentru funcțiile AI. SMTP, GitHub, ngrok și Tailscale sunt necesare doar pentru fluxurile care depind de aceste servicii.

3.2.12.2 Instalarea dependențelor

Listarea 3.4 reproduce comenzile de instalare documentate în proiect.

```
1 python3 -m venv .venv
2 .venv/bin/python -m pip install -r backend/requirements.txt
3
4 cd frontend
5 npm install
6 cd ..
```

Listarea 3.4: Instalarea dependențelor backend și frontend

3.2.12.3 Configurarea mediului

Listarea 3.5 este o variantă sanitizată a configurației din README. Nu conține parole, chei sau adrese private; valorile trebuie înlocuite în mediul local.

```

1 DATABASE_URL=postgresql://USER:PASSWORD@localhost:5432/DB_NAME
2 SECRET_KEY=CHANGE_ME
3 ALGORITHM=HS256
4 ACCESS_TOKEN_EXPIRE_MINUTES=1440
5
6 FRONTEND_URL=http://127.0.0.1:3000
7 BACKEND_URL=http://127.0.0.1:8000
8
9 LOCAL_AI_PROVIDER_NAME=Qwen local (Ollama)
10 LOCAL_AI_BASE_URL=http://127.0.0.1:11434
11 LOCAL_AI_MODEL=qwen2.5-coder:7b
12 LOCAL_AI_AUTO_DETECT=true
13
14 GITHUB_WEBHOOK_SECRET=CHANGE_ME

```

Listarea 3.5: Configurare backend sanitizată

```

1 NEXT_PUBLIC_API_URL=http://127.0.0.1:8000
2 NEXT_PUBLIC_WS_URL=ws://127.0.0.1:8000/ws

```

Listarea 3.6: Configurarea locală a adreselor frontend

3.2.12.4 Migrarea și pornirea

După crearea bazei de date, schema este aplicată cu Alembic, iar aplicația poate fi pornită prin scriptul proiectului. Comenzile verificate în documentația proiectului sunt redate în Listarea 3.7.

```

1 .venv/bin/alembic -c backend/alembic.ini upgrade head
2 ./start.sh

```

Listarea 3.7: Migrarea bazei de date și pornirea aplicației

După pornire, frontend-ul este accesibil local pe portul 3000, backend-ul pe portul 8000, iar documentația Swagger la /docs. Endpoint-ul /health este folosit pentru verificarea disponibilității backend-ului.

3.3 Interfața și utilizarea aplicației

3.3.1 Principii de interfață și navigare

Interfața este organizată ca un workspace deoarece utilizatorul trebuie să treacă frecvent între vedere de ansamblu, planificare, execuție, colaborare și raportare. Sidebar-ul păstrează accesul constant la module, iar zona centrală se schimbă în funcție de pagina selectată. Cardurile sunt folosite pentru sinteze, tabelele pentru

volume mai mari de date, graficele pentru evoluții, iar dialogurile pentru operații care nu justifică părăsirea contextului curent.

Navigarea nu afișează aceleași opțiuni în orice proiect. Metodologia, rolul și starea proiectului influențează paginile și acțiunile disponibile. Un proiect Scrum oferă backlog și sprinturi, în timp ce Kanban pune accent pe board și WIP. Un membru fără permisiuni administrative poate consulta activitatea fără a putea modifica setările. Această adaptare reduce numărul controalelor irelevante și face vizibilă legătura dintre regulile de business și interfață.

Aplicația oferă teme light, dark și system, iar sidebar-ul și dialogurile se adaptează viewport-ului. Figura 3.16 prezintă relația dintre paginile publice, onboarding, selecția proiectului și workspace. Diagrama este importantă deoarece arată că autentificarea nu conduce automat la același ecran: destinația depinde de existența unui proiect, de invitațiile în așteptare și de rolul utilizatorului.

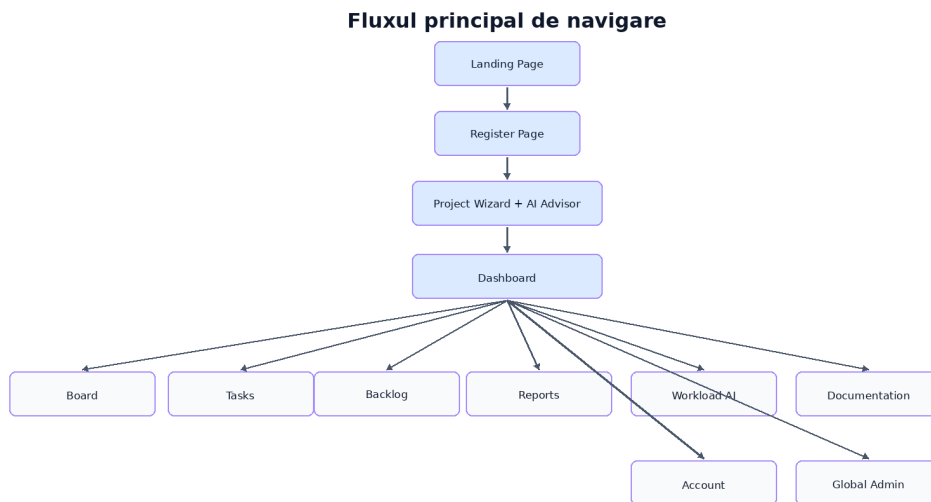


Figura 3.16: Fluxul de navigare între paginile publice, onboarding și workspace

3.3.2 Înregistrare și autentificare

Pagina de înregistrare reprezintă primul punct în care aplicația colectează date personale. Formularul solicită numai informațiile necesare creării contului și oferă validări înainte de trimitere. După salvare, contul trebuie verificat prin e-mail, ceea ce reduce folosirea adreselor introduse greșit și permite ulterior fluxurile de recuperare. Figura 3.17 prezintă modul în care câmpurile și acțiunea principală sunt grupate pentru a păstra un traseu clar.

Create an account
Get started with your AI-powered workspace

Full Name

Email

Password Confirm

[Create Account](#)

Already have an account? [Sign in](#)

Figura 3.17: Pagina de înregistrare și datele necesare creării contului

Pagina de autentificare din Figura 3.18 are rolul de a iniția o sesiune și de a direcționa utilizatorul spre contextul corect. Linkurile către înregistrare și resetarea parolei sunt păstrate în aceeași zonă deoarece reprezintă alternative ale aceluiași flux de acces. În cazul unui cont neverificat, aplicația nu afișează doar o eroare generică, ci îl conduce către pagina de verificare.

SD SDLC Hub
Enter your credentials to access your workspace

Email

Password [Forgot password?](#)

[Sign In](#)

Don't have an account? [Sign up](#)

Figura 3.18: Pagina de autentificare și accesul la recuperarea contului

După autentificare, aplicația verifică proiectele și invitațiile utilizatorului. Un utilizator fără proiect este orientat către onboarding, unul invitat poate accepta

accesul, iar un membru existent poate continua în workspace. Trimiterea reală a mesajelor de verificare și resetare depinde de configurația SMTP și trebuie validată în mediul final de prezentare.

3.3.3 Crearea proiectului

Project Wizard a fost realizat pentru a transforma configurarea inițială într-un flux ghidat. Crearea unui proiect implică mai mult decât alegerea unui nume: trebuie stabilite cheia task-urilor, descrierea, metodologia, rolurile și membrii inițiali. Împărțirea în pași reduce cantitatea de informație prezentată simultan și permite validarea fiecărei decizii înainte de salvare.

Figura 3.19 prezintă ecranul în care utilizatorul avansează de la identitatea proiectului către recomandarea metodologică și configurarea echipei. Interviuul AI este plasat în acest flux deoarece recomandarea este utilă înainte ca regulile proiectului să fie inițializate. Rezultatul include o explicație și poate fi înlocuit manual; aplicația nu condiționează crearea proiectului de acceptarea AI-ului.

Figura 3.19: Project Wizard pentru configurarea identității, metodologiei și echipei

La final, backend-ul creează proiectul, apartenența owner-ului, configurația de workflow, rolurile și invitațiile. Direcționarea către dashboard marchează trecerea de la configurare la utilizarea curentă. Dacă o etapă eșuează, utilizatorul trebuie să primească un mesaj clar și să nu rămână cu impresia că toate datele au fost salvate.

3.3.4 Dashboard, board și task-uri

3.3.4.1 Dashboard

Dashboard-ul este prima pagină a proiectului deoarece utilizatorul are nevoie de o orientare rapidă înainte de a modifica date. Figura 3.20 grupează indicatorii de progres, munca activă, semnalele de livrare și activitatea recentă. Scopul ecranului nu este să reproducă toate rapoartele, ci să evidențieze zonele care necesită atenție și să ofere legături către detalii.

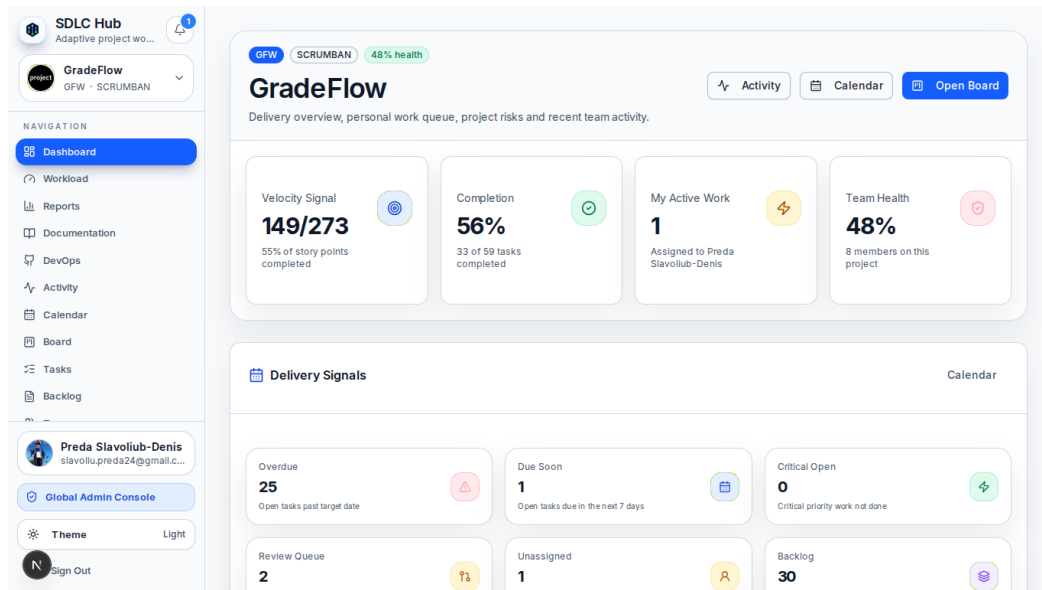


Figura 3.20: Dashboard-ul proiectului și indicatorii utilizați pentru orientarea rapidă

Valorile afișate sunt derivate din task-uri, sprinturi, termene și activitate. Prin urmare, dashboard-ul face vizibilă consecința operațiilor realizate în celelalte module. Un număr ridicat de activități restante sau o distribuție dezechilibrată nu este doar o valoare decorativă, ci un semnal pentru deschiderea paginilor Tasks, Workload sau Reports.

3.3.4.2 Board

Board-ul din Figura 3.21 oferă o reprezentare vizuală a fluxului de lucru. Cardurile sunt grupate pe statusuri, iar mutarea lor permite actualizarea rapidă a stării. În Scrum, board-ul este legat de sprintul activ; în Kanban reprezintă fluxul continuu și poate evidenția limitele WIP; în Scrumban combină planificarea cu flexibilitatea fluxului.

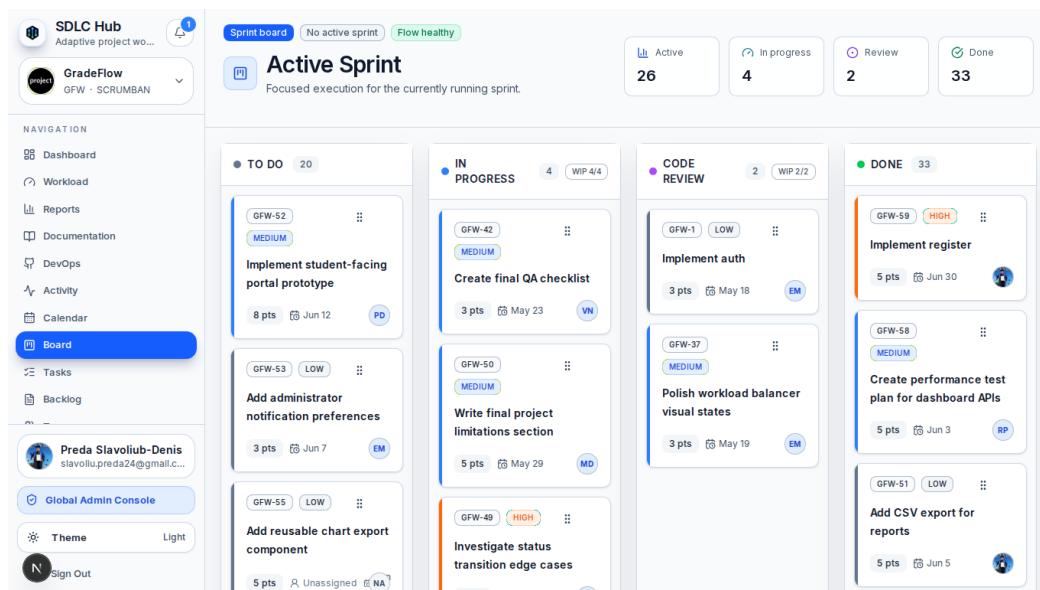


Figura 3.21: Board-ul de lucru și organizarea task-urilor după starea curentă

Interacțiunea drag-and-drop a fost aleasă deoarece schimbarea statusului este una dintre operațiile cele mai frecvente. Totuși, interfața nu modifică local cardul ca și cum operația ar fi suficientă. Cererea este validată de backend, iar un eveniment realtime actualizează ceilalți clienți. Dacă regula proiectului nu permite tranziția, cardul trebuie readus în poziția anterioară și utilizatorul informat.

3.3.4.3 Lista și detaliul task-urilor

Pagina Tasks din Figura 3.22 completează board-ul prin căutare, sortare și filtre după status, prioritate, responsabil, sprint și termen. Această vedere este necesară atunci când proiectul conține mai multe activități decât pot fi analizate eficient pe coloane.

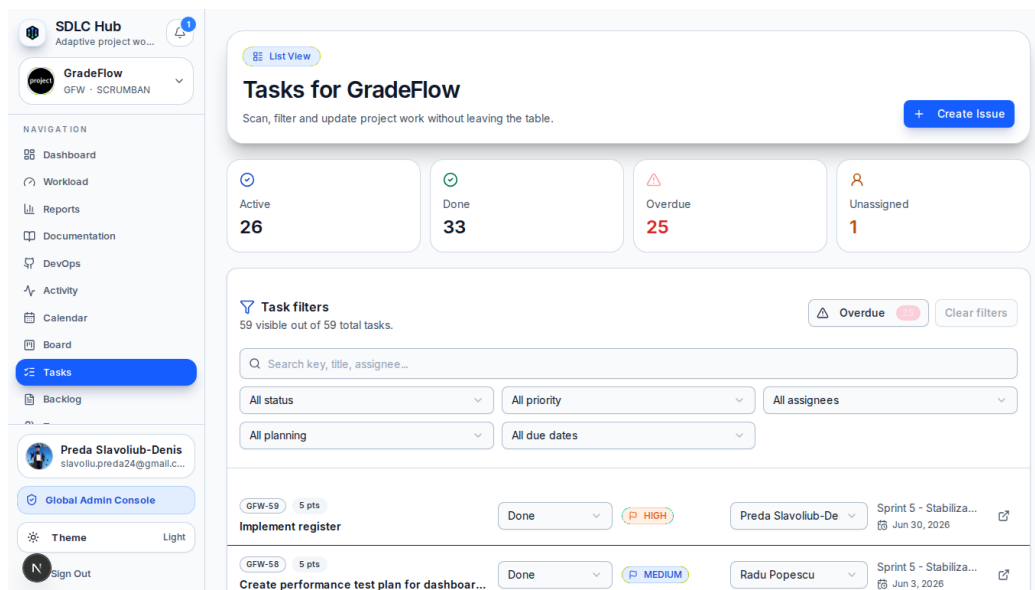


Figura 3.22: Lista filtrabilă utilizată pentru căutarea și analiza task-urilor

Deschiderea unui task conduce la pagina de detaliu, unde utilizatorul găsește descrierea, criteriile de acceptare, estimarea, subtasks, comentariile, istoricul și legăturile DevOps. Concentrarea acestor elemente într-un singur ecran păstrează contextul activității și reduce necesitatea consultării unor instrumente separate. Butoanele sensibile sunt afișate în funcție de permisiuni, iar backend-ul repetă verificarea înainte de salvare.

3.3.5 Utilizarea funcțiilor AI

Funcțiile AI apar în locurile în care există deja context suficient pentru o recomandare. În Project Wizard, răspunsurile despre echipă și proiect sunt folosite pentru recomandarea metodologiei. La crearea sau editarea unui task, descrierea existentă poate fi rafinată, pot fi propuse criterii de acceptare și subtask-uri, iar estimatorul poate genera story points și factori de risc. În Reports, task-urile finalizate devin sursă pentru release notes, iar în Documentation pot fi transformate într-o pagină wiki.

Interfața diferențiază clar textul introdus de utilizator de rezultatul generat. Sugestia este prezentată într-o zonă care permite revizuirea și editarea înainte de aplicare. Această decizie evită salvarea automată a unor formulări eronate și păstrează responsabilitatea asupra cerințelor și estimărilor. Mesajele de încărcare sunt importante deoarece inferența locală poate dura mai mult decât o operație obișnuită de API.

Figura 3.23 prezintă pagina Workload. Graficul sau distribuția activității oferă baza factuală, iar recomandarea AI explică de ce ar putea fi utilă o redistribuire. Sugestia nu modifică automat responsabilul deoarece numărul de task-uri și story

points nu surprinde complet experiența, disponibilitatea sau specializarea membrilor.

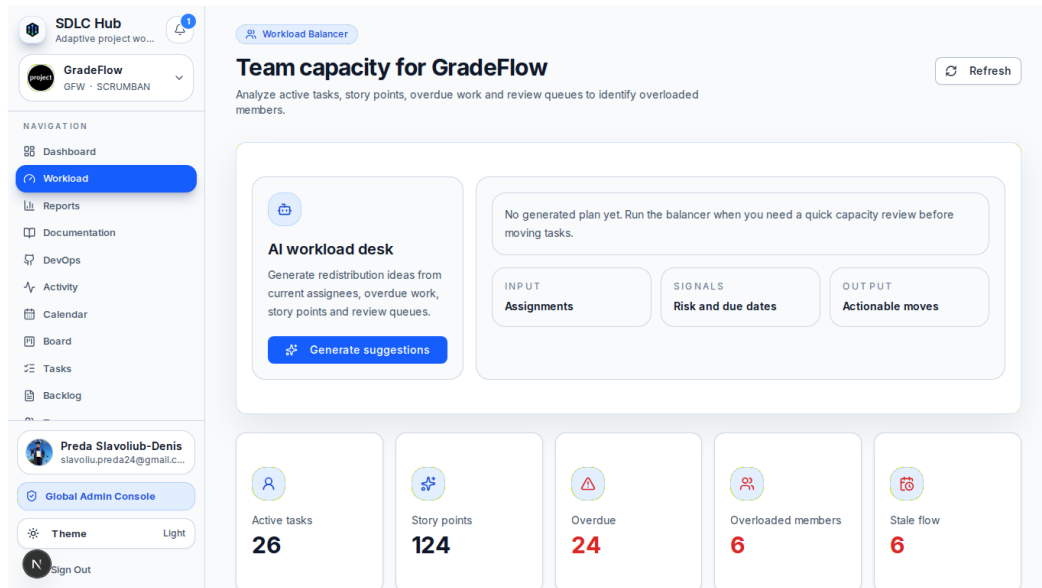


Figura 3.23: Pagina Workload și recomandările explicate pentru echilibrarea activității

Atunci când providerul nu este disponibil, interfața trebuie să indice eroarea sau folosirea fallback-ului, fără a bloca editarea manuală. În acest mod, AI-ul îmbunătățește fluxul, dar nu devine o condiție pentru funcționarea de bază a proiectului.

3.3.6 Rapoarte și documentație

Pagina Reports din Figura 3.24 transformă datele operaționale în informații care pot fi interpretate de echipă și de coordonatori. Metricele nu sunt identice pentru toate metodologiile. Velocity și burndown au sens în raport cu sprinturile, în timp ce lead time, cycle time, cumulative flow și WIP descriu mai bine fluxul Kanban. Selectarea metricilor în funcție de metodologie evită afișarea unor grafice care nu corespund modului de lucru.

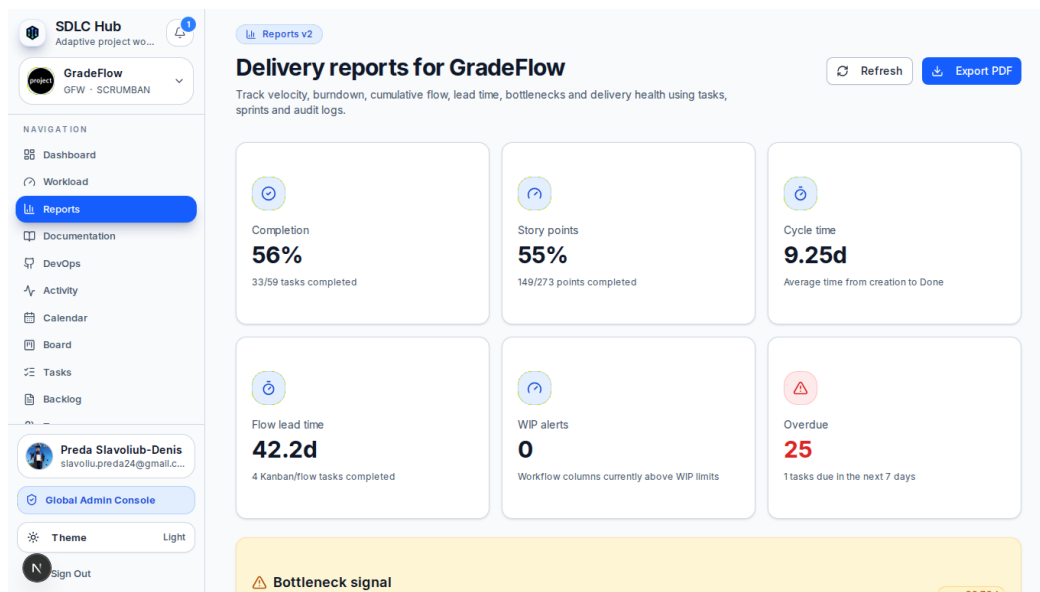


Figura 3.24: Pagina Reports și metricele utilizate pentru evaluarea livrării

Exportul PDF permite păstrarea unei versiuni a raportului în afara aplicației. Acesta este util pentru prezentări, arhivare și comunicarea cu stakeholderii, dar nu înlocuiește datele live. Generarea release notes folosește activitățile finalizate pentru a propune o descriere a livrării, care trebuie verificată înainte de publicare.

Pagina Documentation din Figura 3.25 funcționează ca wiki al proiectului. Structura pe pagini și revizii permite păstrarea explicațiilor care nu se potrivesc într-un comentariu de task. Generarea unei pagini dintr-o activitate finalizată reduce riscul ca informația tehnică să rămână numai în istoricul implementării.

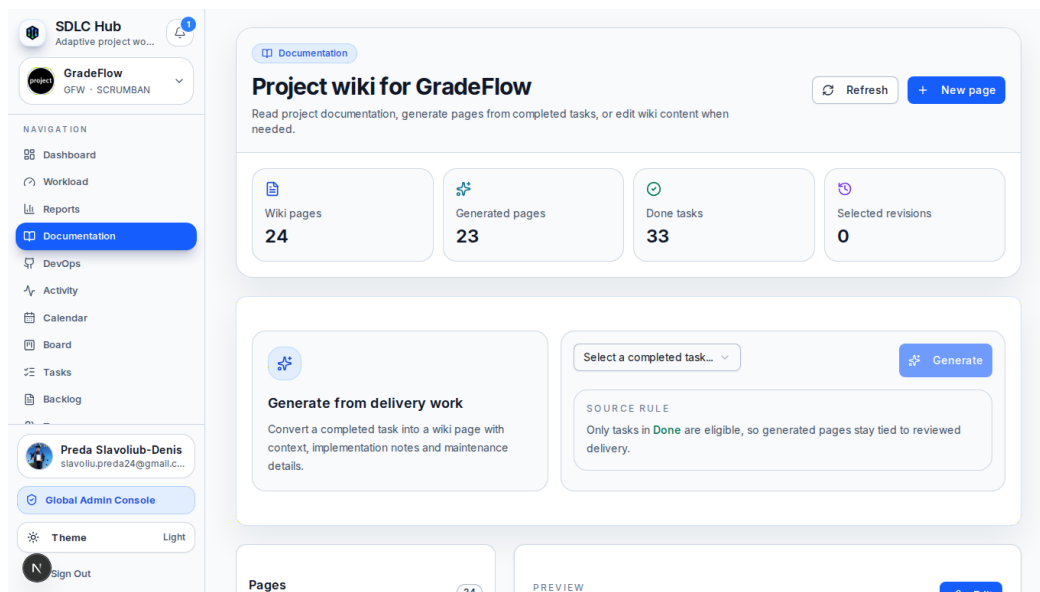


Figura 3.25: Pagina de documentație internă și organizarea cunoștințelor proiectului

3.3.7 Contul utilizatorului și administrarea globală

Pagina Account din Figura 3.26 reunește datele personale, avatarul, preferințele, sesiunile active și jurnalul de securitate. Separarea acestei pagini de setările proiectului este necesară deoarece profilul și sesiunile aparțin utilizatorului, în timp ce rolurile și integrarea GitHub aparțin unui proiect. Posibilitatea revocării unei sesiuni oferă control atunci când un dispozitiv nu mai este utilizat.

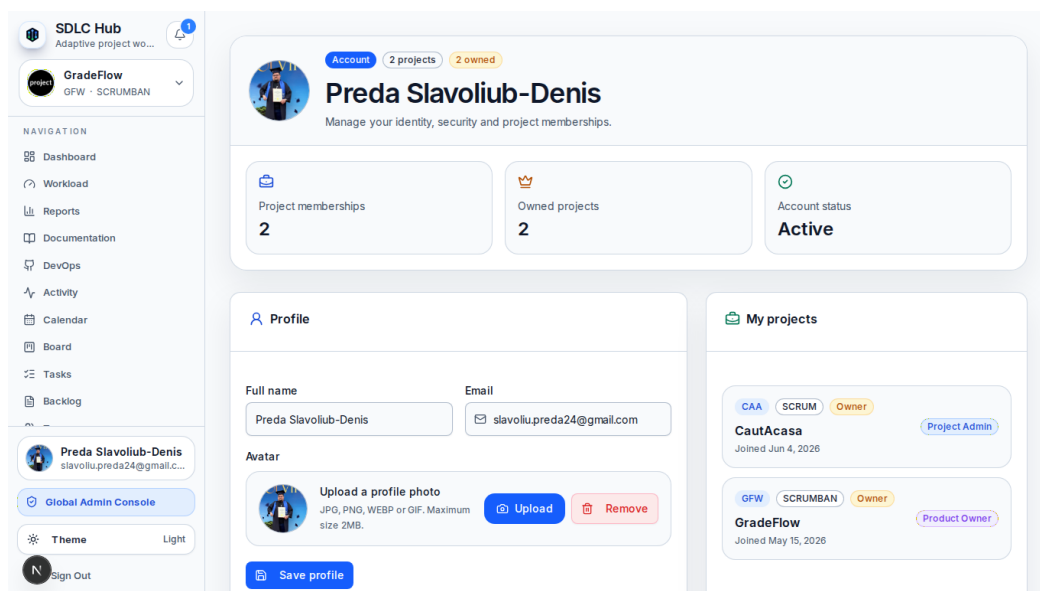


Figura 3.26: Pagina Account pentru profil, sesiuni și jurnalul de securitate

Global Admin Console din Figura 3.27 este accesibilă numai actorului global și are o navigare separată de workspace. Ecranul prezintă utilizatori, proiecte, suport, consum AI și erori de sistem. Această separare face clar faptul că un Project Owner nu primește automat drepturi asupra întregii platforme.

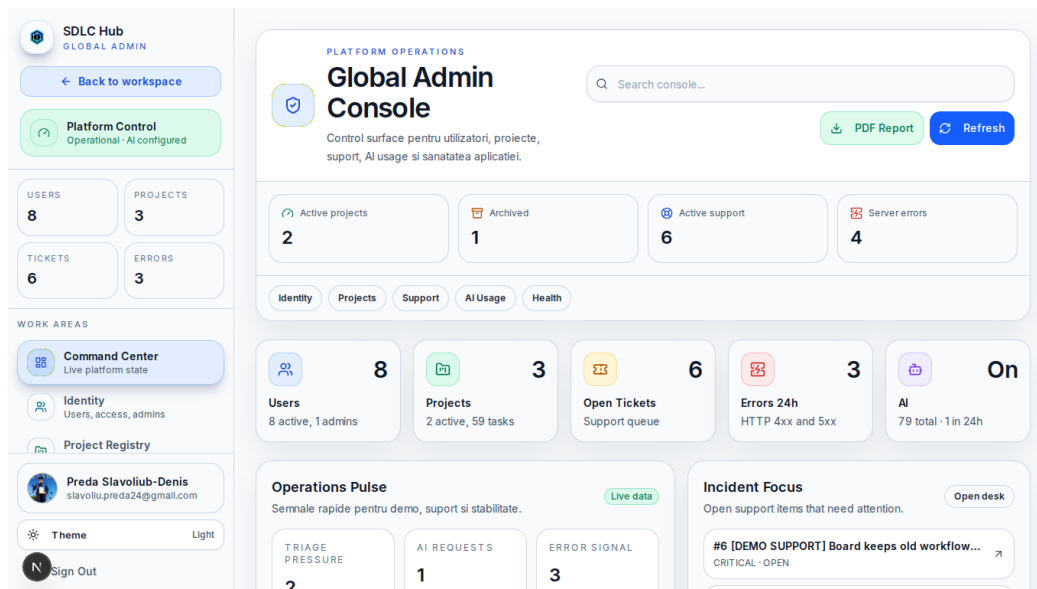


Figura 3.27: Consola Global Admin și funcțiile de supraveghere a platformei

Tichetele de suport oferă un canal intern pentru raportarea problemelor, iar jurnalul erorilor HTTP ajută administratorul să identifice rutele și momentele în care apar răspunsuri problematice. AI usage oferă o vedere operațională asupra funcțiilor care pot avea durate și costuri diferite. Ecranul demonstrează că administrarea unei platforme nu se reduce la ștergerea utilizatorilor, ci include observarea stării și a modului de utilizare.

3.3.8 Manual scurt de utilizare

Utilizarea sistemului începe prin pornirea backend-ului, frontend-ului și bazei de date, urmată de verificarea endpoint-ului `/health`. Utilizatorul creează un cont sau se autentifică, apoi este direcționat către onboarding, invitațiile în așteptare sau un proiect existent. Pentru un proiect nou, Project Wizard colectează datele de bază, recomandă o metodologie și permite configurarea rolurilor și invitațiilor.

După intrarea în workspace, dashboard-ul oferă orientarea inițială. Activitățile sunt create în pagina Tasks sau din dialogurile disponibile, apoi sunt planificate în backlog și sprint ori introduse direct în fluxul Kanban. Board-ul este utilizat pentru urmărirea stării, iar pagina de detaliu pentru descriere, subtask-uri, comentarii, estimări și audit. Funcțiile AI pot fi solicitate în aceste momente, însă rezultatul trebuie citit și ajustat înainte de salvare.

Calendarul și notificările sunt consultate pentru evenimente și schimbări care necesită atenție. Workload și Reports oferă informații despre distribuția activității și progres. Integrarea GitHub se configurează numai de un utilizator autorizat și necesită repository, secret și webhook valide. După finalizarea activităților, echipa poate genera release notes, exporta raportul și actualiza documentația internă. Profilul, sesiunile și preferințele se gestionează din Account, iar funcțiile globale sunt disponibile exclusiv în consola de administrare.

3.3.9 Exemplu complet de utilizare

Owner-ul creează un proiect Scrum și invită un Developer și un QA. Product Owner-ul introduce o cerință scurtă, folosește Spec Refiner și verifică rezultatul. Task-ul este estimat, adăugat într-un sprint și mutat pe board. Developer-ul leagă un commit prin cheia task-ului, QA adaugă un comentariu, iar echipa mută task-ul în *Done*. Sistemul poate genera documentație și release notes, iar Reports reflectă livrarea. Acest scenariu folosește aceleași date în mai multe module și demonstrează trasabilitatea.

Capitolul 4

Discutarea rezultatelor obținute

4.1 Testare și rezultate

4.1.1 Strategia de testare

Strategia de testare urmărește verificarea sistemului pe mai multe niveluri, deoarece o aplicație full-stack poate afișa corect o pagină chiar dacă regulile backend-ului sunt greșite sau poate răspunde corect prin API fără ca fluxul din browser să fie utilizabil. Testele API verifică rutele, validările, permisiunile și structura răspunsurilor. Testele funcționale urmăresc scenarii complete, de la autentificare și selectarea proiectului până la task-uri, rapoarte și administrare.

Interfața este verificată pe proiecte Playwright desktop și mobile, iar testele responsive urmăresc navigarea și layout-ul la viewport redus. Funcțiile AI sunt testate separat deoarece au durate mai mari, pot utiliza fallback și trebuie să întoarcă rezultate controlabile. Integrarea GitHub, exporturile și consola Global Admin sunt incluse în suite dedicate. Serviciile externe care depind de credențiale sau infrastructură reală sunt completate prin verificări manuale.

Playwright permite automatizarea interacțiunilor în browser și a cererilor API [Mic26]. Suitele proiectului sunt organizate în `frontend/tests/e2e`, iar raportul HTML păstrează durata și rezultatul fiecărui scenariu.

4.1.2 Mediul de testare

Raportul disponibil a fost generat la 9 iulie 2026, ora 09:02:50. El folosește proiectele Playwright `chromium-desktop` și `chromium-mobile`. Durata agregată raportată este 190,70 secunde. Rezultatele sunt dependente de baza locală, credențialele de test și disponibilitatea serviciilor.

4.1.3 Testarea API

Testele API acoperă autentificarea, utilizatorul curent, sesiuni, security log, onboarding, invitații și notificări. Pentru proiecte sunt verificate membrii, rolurile, permisiunile, auditul, dashboard-ul, rapoartele, workload-ul, setările AI și tranziția metodologiei.

Work engine-ul este testat pentru task-uri, subtasks, comentarii, activity, sprinturi, calendar, disponibilitate, documentație și echipe. Zona DevOps/admin verifică integrarea GitHub, evenimentele, pull request-urile, ngrok, overview-ul global, utilizatorii, proiectele, tichetele, erorile și exporturile.

4.1.4 Testarea UI desktop și mobilă

Testele UI verifică paginile publice și de autentificare, workspace-ul, task detail și consola Global Admin. Sunt deschise Dashboard, Tasks, Board, Backlog, Calendar, Activity, Workload, Reports, Documentation, DevOps, Team, Support, Settings, Account și Notifications. Proiectul mobil verifică sidebar-ul și accesibilitatea fluxurilor pe viewport redus.

4.1.5 Testarea funcționalităților AI

Suita AI verifică recomandarea metodologiei, sugestiile de roluri, generarea și rafinarea specificațiilor, Poker Estimator, Workload Balancer, configurarea providerului, release notes și documentația automată. Testele nu urmăresc numai existența unui răspuns, ci și faptul că endpoint-ul rămâne controlat atunci când providerul este indisponibil sau când intrarea este incompletă.

Cele mai lente scenarii din raport sunt asociate generării AI. Workload Balancer are aproximativ 17,16 secunde, iar Spec Refiner și Methodology Advisor ajung la aproximativ 5,9 secunde în mediul raportat. Aceste durate justifică indicatorii de încărcare și mesajele de stare din interfață. Ele nu trebuie interpretate ca benchmark universal, deoarece depind de dispozitiv, model, rețea și încărcarea serviciului local.

Validarea funcțională nu poate demonstra corectitudinea semantică a fiecărei sugestii. De aceea, testele sunt completate prin verificarea manuală a structurii rezultatului și prin regula de produs conform căreia utilizatorul poate edita sau respinge ieșirea înainte de salvare.

4.1.6 Raportul Playwright

Figura 4.1 prezintă sumarul raportului. Imaginea este inclusă din folderul existent al proiectului.

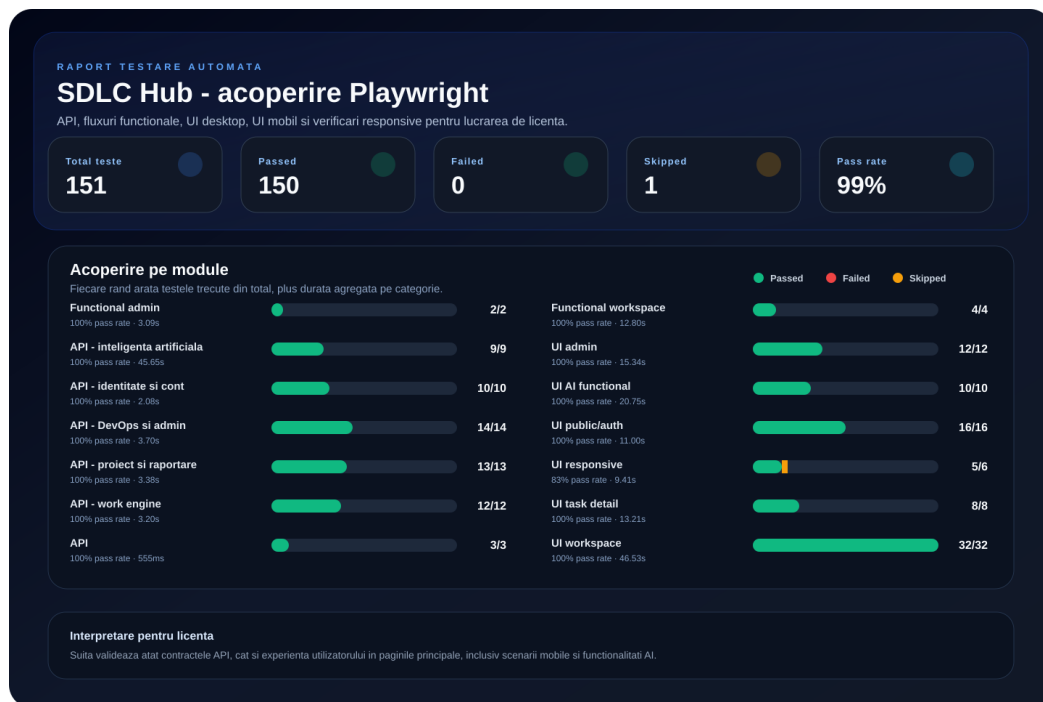


Figura 4.1: Rezumatul rulării Playwright

Tabelul 4.1 sintetizează rezultatele.

Indicator	Valoare
Total teste	151
Trecute	150
Eșuate	0
Omise	1
Rată de trecere raportată	99%
Durată agregată	190,70 s
Categorii	14
Proiecte browser	desktop și mobile

Tabela 4.1: Rezultatele raportului Playwright existent

4.1.7 Interpretarea rezultatelor

Absența testelor eșuate în raport arată că scenariile automatizate au funcționat în mediul respectiv. Testul omis trebuie identificat înaintea predării pentru a stabili dacă omiterea este intenționată sau cauzată de o dependență. Rezultatele nu demonstrează lipsa defectelor, ci doar comportamentul pentru datele și scenariile testate.

Acoperirea desktop și mobile susține afirmația că interfața este responsive. Testele API oferă încredere în validări și permisiuni, iar testele funcționale verifică

legătura dintre module. Timpii mai mari ai funcțiilor AI indică o zonă de optimizare și necesitatea unui feedback vizual clar în UI.

4.1.8 Limitările testării

Raportul analizat nu a fost rerulat în cadrul redactării acestei versiuni, iar valorile prezentate corespund execuției din 9 iulie 2026. Livrarea e-mailurilor printr-un cont SMTP real, primirea webhook-urilor dintr-un repository configurat prin ngrok și conectivitatea Ollama prin Tailscale trebuie validate în mediul exact al prezentării. Aceste fluxuri depind de credențiale, firewall, adrese și servicii care nu pot fi deduse numai din cod.

Testele raportate nu acoperă încărcare concurentă mare, atacuri avansate, disponibilitate ridicată sau compatibilitatea cu toate browserele. Nici calitatea semantică a textelor AI nu poate fi redusă la un rezultat de tip passed sau failed. Pentru demonstrația finală este necesar un checklist care să includă serviciile externe, conținuturile de test, modelul Ollama și repository-ul folosit pentru evenimente GitHub.

4.2 Evaluarea soluției și discutarea rezultatelor

4.2.1 Raportarea la obiectivele lucrării

Rezultatele obținute trebuie evaluate în raport cu obiectivele formulate în introducere, nu numai prin numărul de pagini sau de endpoint-uri implementate. Obiectivul de susținere a metodologiilor diferite este reflectat de configurația care activează backlog, sprinturi, story points și rapoarte pentru Scrum și Scrumban, respectiv flux continuu și limite WIP pentru Kanban. Tranzițiile controlate arată că metodologia influențează datele și regulile, nu este doar o etichetă vizuală.

Obiectivul de centralizare este susținut prin legătura dintre proiecte, task-uri, comentarii, audit, calendar, documentație, rapoarte și GitHub. Un task finalizat poate apărea în raport, poate fi asociat cu un commit sau pull request și poate deveni sursă pentru documentație. Această reutilizare a acelorași date reduce pierderea contextului, care a reprezentat problema principală formulată la începutul lucrării.

Integrarea AI a fost realizată în activități concrete și păstrează decizia finală la utilizator. Recomandarea metodologiei, rafinarea specificației, estimarea, analiza riscului, echilibrarea încărcării și generarea documentației sunt conectate la datele proiectului. Implementarea local-first prin Ollama demonstrează că aceste fluxuri pot funcționa fără trimiterea implicită a conținutului către un serviciu cloud, deși performanța depinde de infrastructura locală.

4.2.2 Semnificația rezultatelor de testare

Rezultatul de 150 de teste trecute, niciun test eșuat și un test omis în rularea raportată oferă încredere în scenariile acoperite. Valoarea sa este mai mare decât o verificare manuală izolată deoarece include API, interfață desktop, interfață mobilă,

funcții AI și module administrative. Totuși, acest rezultat nu demonstrează absența defectelor și nu poate fi generalizat la orice configurație.

Duratele mai mari ale testelor AI evidențiază o caracteristică importantă a soluției: operațiile de inferență au un profil diferit de operațiile CRUD. Interfața trebuie să comunice faptul că solicitarea este în curs și să permită revenirea controlată la editare. În același timp, fallback-ul determinist și separarea funcțiilor AI de managementul de bază împiedică indisponibilitatea modelului să blocheze proiectul.

Acoperirea mobilă confirmă că paginile principale pot fi accesate pe viewport redus, dar nu schimbă faptul că board-ul, rapoartele și configurarea complexă sunt mai potrivite pentru desktop. Rezultatul trebuie interpretat ca accesibilitate responsive, nu ca echivalență completă între experiența mobilă și cea desktop.

4.2.3 Raportarea la soluțiile existente

Jira oferă un ecosistem matur, configurare extinsă și raportare avansată, iar această lucrare nu pretinde că SDLC Hub îl înlocuiește într-un mediu enterprise. Contribuția proprie este integrarea într-un prototip coerent a metodologiei adaptive, AI-ului local, documentației, auditului și administrării globale. Față de Trello, SDLC Hub oferă mai multe reguli de proiect, permisiuni și trasabilitate, dar are o interfață mai complexă și o maturitate mai redusă. Față de GitHub Projects, soluția extinde managementul dincolo de repository și include calendar, rapoarte, documentație și roluri de proiect, însă integrarea GitHub depinde de configurarea webhook-ului.

Comparația arată că avantajul principal nu este existența izolată a board-ului sau a AI-ului, deoarece produse comerciale oferă deja astfel de funcții. Diferențiatorul este modul în care aceste componente sunt legate într-o soluție local-first și adaptată proiectelor Scrum, Kanban și Scrumban. Dezavantajele sunt lipsa validării la scară mare, dependența de configurația locală și numărul mai redus de integrări externe.

4.2.4 Valoarea practică și limitele soluției

SDLC Hub demonstrează funcționarea unei arhitecturi complete care include autentificare, autorizare, persistență relațională, comunicare realtime, integrare DevOps, raportare și AI. Pentru o lucrare preponderent practică, valoarea constă atât în funcțiile vizibile, cât și în regulile care păstrează consistența datelor între module.

Limitele rămân relevante. E-mailul, webhook-urile și AI-ul local depind de servicii configurate extern. Aplicația nu a fost testată pentru trafic enterprise, iar securitatea necesită măsuri suplimentare înainte de expunere publică. Rezultatele AI pot fi incorecte, motiv pentru care sistemul nu le aplică automat. Aceste limite nu anulează rezultatele, ci definesc domeniul în care concluziile lucrării sunt valabile.

Capitolul 5

Concluzii și direcții viitoare

5.1 Sinteza rezultatelor

Scopul lucrării a fost realizarea unei platforme web de management al proiectelor software, cu suport pentru metodologii diferite și funcționalități asistate de AI. Obiectivul a fost atins prin dezvoltarea SDLC Hub, care oferă fluxuri pentru conturi, proiecte, task-uri, sprinturi, board, roluri, permisiuni, audit, calendar, notificări, documentație, rapoarte, GitHub, realtime și administrare globală.

Cea mai importantă contribuție este tratarea metodologiei ca parte activă a sistemului. Aplicația modifică paginile, regulile, câmpurile și rapoartele în funcție de Scrum, Kanban sau Scrumban, iar schimbarea metodologiei este controlată.

A doua contribuție este integrarea AI local-first. Funcțiile AI sunt conectate la Project Wizard, task-uri, workload, reports și documentation. Rezultatele sunt editabile, iar utilizatorul păstrează responsabilitatea deciziei.

5.2 Ce a funcționat și dificultăți întâmpinate

Separarea pe frontend, backend și bază de date a permis dezvoltarea modulară. Auditul și modelul relațional au oferit trasabilitate, iar WebSocket a conectat actualizările dintre pagini. Integrarea testelor Playwright a produs un raport repetabil pentru API, UI și responsive.

Dificultățile principale au fost coordonarea regulilor dintre metodologii, păstrarea permisiunilor consecvente, tratarea serviciilor externe și integrarea AI fără blocarea fluxurilor principale. Tranzițiile metodologice au fost abordate prin preview și validări. Indisponibilitatea AI este tratată pentru unele funcții prin fallback. Secretele sunt scoase din cod și gestionate prin configurare.

5.3 Probleme rămase deschise

Serviciile SMTP, GitHub/ngrok și Ollama/Tailscale trebuie validate în mediul final. Aplicația nu a fost evaluată pentru trafic enterprise, atacuri complexe sau disponibilitate ridicată. Fragmentele suplimentare pentru serviciul AI și testele Playwright

trebuie inserate numai după verificarea fișierelor finale, pentru a evita folosirea unei versiuni depășite.

5.4 Direcții viitoare

O direcție importantă este extinderea componentei operaționale prin rularea și păstrarea regresiilor direct din consola Global Admin. Această funcție ar permite administratorului să coreleze o versiune a aplicației cu rezultatele testelor și cu erorile HTTP observate după publicare. Pentru proiectele cu volume mari de date sunt necesare virtualizarea listelor, căutare cu debounce și un cache controlat pentru dashboard, rapoarte și workload.

Integrarea DevOps poate fi extinsă către GitLab și Bitbucket, iar calendarul intern poate comunica cu servicii externe. Rapoartele pot deveni configurabile și pot include analiză cross-project pentru organizații care administrează mai multe produse. Din punct de vedere al securității, sunt necesare politici mai stricte pentru proiectele arhivate, rotația secretelor, rate limiting și audit operațional avansat.

Componenta AI poate fi îmbunătățită prin colectarea feedback-ului asupra sugestiilor și prin folosirea documentației interne într-un mecanism RAG. Astfel, răspunsurile ar putea utiliza terminologia și deciziile proiectului, nu doar descrierea curentă a task-ului. Orice extindere trebuie să păstreze principiile soluției: rezultatul este explicat, verificabil și acceptat de utilizator înainte de a produce modificări.

Bibliografie

- [Atl26a] Atlassian. Jira software features. <https://www.atlassian.com/software/jira/features>, 2026. Accesat la 11 iulie 2026.
- [Atl26b] Atlassian. RoVo in jira: Ai features. <https://www.atlassian.com/software/jira/ai>, 2026. Accesat la 11 iulie 2026.
- [Atl26c] Atlassian. Trello guide. <https://trello.com/guide>, 2026. Accesat la 11 iulie 2026.
- [Atl26d] Atlassian Support. Use ai to help write or edit content in trello. <https://support.atlassian.com/trello/docs/use-atlassian-intelligence-to-help-write-or-edit-content/>, 2026. Accesat la 11 iulie 2026.
- [dnd26] dnd kit. dnd kit documentation. <https://docs.dndkit.com/>, 2026. Accesat la 11 iulie 2026.
- [Fas26] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026. Accesat la 11 iulie 2026.
- [Git26a] GitHub Docs. About projects. <https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>, 2026. Accesat la 11 iulie 2026.
- [Git26b] GitHub Docs. Using github copilot to create or update issues. <https://docs.github.com/en/copilot/how-tos/copilot-on-github/copilot-for-github-tasks/use-copilot-to-create-or-update-issues>, 2026. Accesat la 11 iulie 2026.
- [Git26c] GitHub Docs. Validating webhook deliveries. <https://docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries>, 2026. Accesat la 11 iulie 2026.
- [Git26d] GitHub Docs. Webhooks documentation. <https://docs.github.com/en/webhooks>, 2026. Accesat la 11 iulie 2026.

- [IBM26] IBM. What is three-tier architecture? <https://www.ibm.com/topics/three-tier-architecture>, 2026. Accesat la 11 iulie 2026.
- [JBS15] Michael B. Jones, John Bradley, and Nat Sakimura. RFC 7519: Json web token (JWT). <https://www.rfc-editor.org/rfc/rfc7519>, 2015.
- [LGJO17] Howard Lei, Farnaz Ganjeizadeh, Pradeep Kumar Jayachandran, and Pinar Ozcan. A statistical analysis of the effects of scrum and kanban on software development projects. *Robotics and Computer-Integrated Manufacturing*, 43:59–67, 2017.
- [MDN26] MDN Web Docs. The websocket api. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API, 2026. Accesat la 11 iulie 2026.
- [Met26] Meta Open Source. React documentation. <https://react.dev/>, 2026. Accesat la 11 iulie 2026.
- [Mic26] Microsoft. Playwright documentation. <https://playwright.dev/>, 2026. Accesat la 11 iulie 2026.
- [MKA⁺21] Yasir Mahmood, Nazri Kama, Azri Azmi, Ahmad Salman Khan, and Mazlan Ali. Software effort estimation accuracy prediction of machine learning techniques: A systematic performance evaluation. <https://arxiv.org/abs/2101.10658>, 2021.
- [MUAZ⁺24] Mariyam Mahboob, Mohammed Rayyan Uddin Ahmed, Zoiba Zia, Mariam Shakeel Ali, and Ayman Khaleel Ahmed. Future of artificial intelligence in agile software development. <https://arxiv.org/abs/2408.00703>, 2024.
- [Oll26] Ollama. Ollama documentation. <https://ollama.com/>, 2026. Accesat la 11 iulie 2026.
- [pmn26] pmndrs. Zustand documentation. <https://zustand.docs.pmnd.rs/>, 2026. Accesat la 11 iulie 2026.
- [Pos26] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2026. Accesat la 11 iulie 2026.
- [Rad26] Radix UI. Radix ui documentation. <https://www.radix-ui.com/>, 2026. Accesat la 11 iulie 2026.
- [Rec26] Recharts. Recharts documentation. <https://recharts.org/>, 2026. Accesat la 11 iulie 2026.

- [SQL26a] SQLAlchemy. Alembic documentation. <https://alembic.sqlalchemy.org/en/latest/>, 2026. Accesat la 11 iulie 2026.
- [SQL26b] SQLAlchemy. Ssqlalchemy documentation. <https://docs.sqlalchemy.org/>, 2026. Accesat la 11 iulie 2026.
- [SS20] Ken Schwaber and Jeff Sutherland. The scrum guide: The definitive guide to scrum. <https://scrumguides.org/scrum-guide.html>, 2020. Accesat la 11 iulie 2026.
- [Tai26] Tailwind Labs. Tailwind css documentation. <https://tailwindcss.com/docs>, 2026. Accesat la 11 iulie 2026.
- [Tip26] Tiptap. Tiptap documentation. <https://tiptap.dev/docs>, 2026. Accesat la 11 iulie 2026.
- [Ver26] Vercel. Next.js documentation. <https://nextjs.org/docs>, 2026. Accesat la 11 iulie 2026.
- [VY20] Daniel Vacanti and Yuval Yeret. The kanban guide. <https://kanbanguides.org/>, 2020. Accesat la 11 iulie 2026.
- [ZDA⁺25] Mohammad Amin Zadenoori, Jacek Dąbrowski, Waad Alhoshan, Liping Zhao, and Alessio Ferrari. Large language models (LLMs) for requirements engineering (RE): A systematic literature review. <https://arxiv.org/abs/2509.11446>, 2025.